

SMART RIDE TIET

Capstone Project Report

End-Semester Evaluation

Submitted by:

Prabhdeep Singh (2024010078)

Shamandeep Singh (2024010095)

Purva (2024010082)

Nupur (2024010075)

MCA Final Year

Group No: 6

Under the Mentorship of

Dr Karamjeet Singh



THAPAR INSTITUTE

OF ENGINEERING & TECHNOLOGY

(Deemed to be University)

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala, 147001

24 December, 2025

ABSTRACT

SmartRideTIET is a modern web application created to improve intra-campus mobility and personal convenience during daily travel across Thapar Institute of Engineering and Technology (TIET). It provides users with a reliable platform where they can access verified campus e-rickshaws and receive timely assistance during urgent transitions. In addition, the system aims to reduce the delays and friction people face while walking alone or waiting at physical stands, especially between distant hostels and academic venues. By integrating real-time telemetry, calibrated 2D landmark coordinates, and smart decision-making, the application helps students and faculty choose campus transit that is not only faster but also affordable and safe.

The app allows students to register, view active campus rickshaws, and start an on-demand booking whenever they need to commute across the 250-acre perimeter. Moreover, techniques such as calibrated landmark-to-landmark tracking and automated dispatch queuing ensure that an available rickshaw can reach the passenger whenever required. When a ride is booked, the application instantly verifies the trip with a 4-digit security One-Time Password (OTP), thereby preventing unauthorized boarding and improving reliability. This seamless communication builds a sense of trust and accountability.

Furthermore, SmartRideTIET focuses on user experience by providing a clean interface that is easy to navigate across mobile devices. It connects essential micro-mobility functionalities—landmark search, live radar monitoring, cashless virtual wallet settlement backed by Razorpay, and administrative fleet oversight—into one unified system. Through these features, the app not only enhances daily convenience but also supports users in making informed decisions during travel. Ultimately, we conclude that SmartRideTIET offers a practical yet effective solution for students and campus drivers seeking enhanced efficiency, safety, and confidence in everyday campus mobility.

DECLARATION

We hereby declare that the design principles and working prototype model of the project entitled “SmartRideTIET: Intelligent Intra-Campus Micro-Mobility Platform” is an authentic record of our own carried out in the Computer Science and Engineering Department TIET Patiala under the guidance of Dr Karamjeet Singh during the 3rd Semester of MCA (2025).

Date: 24/10/2025

Roll No	Name	Signature
2024010078	Prabhdeep Singh	
2024010095	Shamandeep Singh	
2024010075	Nupur	
2024010082	Purva	

Counter Signed By:

Faculty Mentor:

Dr. Karamjeet Singh

Assistant Professor, CSED

TIET, Patiala

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our mentor Dr Karamjeet Singh. His invaluable guidance and technical knowledge have been crucial to our project's success. We are thankful to the Head of the Computer Applications Department, the entire faculty, and staff of the Computer Applications Department, and our friends who provided their time and support in completing this project. We thank all those who contributed directly or indirectly to this project.

Date: 24/10/2025

Roll No	Name	Signature
2024010078	Prabhdeep Singh	
2024010095	Shamandeep Singh	
2024010075	Nupur	
2024010082	Purva	

INDEX / TABLE OF CONTENTS

1. Introduction	1-7
1.1 Technical Terminology	
1.2 Problem Statement	
1.3 Goals	
1.4 Need Analysis	
1.5 Research Gaps	
1.6 Problem Definition and Scope	
1.7 Assumptions and Constraints	
2. Literature Survey	8-13
2.1 Theory Associated with Problem Area	
2.2 Existing Systems and Solutions	
2.3 The Problem That Has Been Identified	
2.4 Survey of Tools and Technologies Used	
2.5 Standards Used	
3. Methodology	14-16
3.1 Data Collection and Preprocessing	
3.2 Route Generation	
3.3 Crime Analysis and Hotspot Detection	
3.4 Feature Extraction	
3.5 Safety Score Prediction (Machine Learning)	
3.6 Route Ranking and Recommendation	
3.7 Emergency and Safety Features	
3.8 Authentication and User Management	
3.9 Frontend Implementation	
3.10 Testing and Validation	
4. System Design	17-24
4.1 Block Diagram of System Workflow	
4.2 Work Breakdown Structure	
4.3 Class Diagram	
4.4 Use Case Diagram	

4.5 Sequence Diagram

4.6 Project Implementation	
5. Model Discussion	25-33
5.1 Abstract	
5.2 Introduction	
5.3 Problem Definition	
5.4 Working of Machine Learning Model	
5.5 Conclusion	
6. Cost Analysis	34-35
7. Conclusion	36
8. References	37

LIST OF FIGURES

1. System Design

Figure 4.1 Block Diagram	17-20
Figure 4.2 Work Breakdown Structure	
Figure 4.3 Class Diagram	
Figure 4.4 Use Case diagram	
Figure 4.5 Sequence Diagram	

2. Project Implementation

Figure 4.6.1 Sign in Page	21-24
Figure 4.6.2 Home Page	
Figure 4.6.3 Location Requesting Message	
Figure 4.6.4 Safes Routes Page	
Figure 4.6.5 Personal Safety Page	
Figure 4.6.6 Safety Check Page	
Figure 4.6.7 My Profile Page	
Figure 4.6.8 Emergency Contact Page	
Figure 4.6.9 List of Emergency Contacts	
Figure 4.6.10 Live Sharing	
Figure 4.6.11 Multiple Routes for destination	
Figure 4.6.12 Safest Route Selection	
Figure 4.6.13 Emergency SOS	
Figure 4.6.14 Safety Audit	

3. Data Visualization

Figure 5.1 Crimes By hour of Day	30-33
Figure 5.2 Average Severity by Crime Category	
Figure 5.3 Crimes per years	
Figure 5.4 Crime Intensity Hexbin Map	
Figure 5.5 HeatMap(Longitude, Latitude)	
Figure 5.6 Rolling Average of Crimes	
Figure 5.7 Scatter Plot Crime Location	

Figure 5.8 Bar Chart Common Crime Categories

Figure 5.9 Scatter Plot Geographic Distribution

Figure 5.10 Box plot Distribution on Crime Severity

LIST OF TABLES

Table 1: SafeSteps Competitor Comparison Table	10-11
Table 2: Development Resource Cost	34-35
Table 3: Cloud and API Usage	35
Table 4: Maintenance and Future Scaling	35

CHAPTER -1

INTRODUCTION

SmartRideTIET is a web-based mobile application designed to make campus travel easier, especially for students, faculty, and campus visitors at Thapar Institute of Engineering and Technology (TIET). The app goes beyond ordinary navigation by not only showing campus pathways, but also by connecting riders directly with verified campus e-rickshaws and optimizing intra-campus micro-mobility. It does this by combining campus GPS coordinates, landmark mapping, and machine learning analytics.

The system works through three main parts:

- A mobile web app (React.js, Vite) where users can explore 60+ campus landmarks, request rides, view trip fares, and track drivers.
- A backend orchestration service (Node.js, Express) that manages role-based authentication, ride state transitions, wallet transactions, and administrative oversight.
- A machine learning service (FastAPI, Python) that analyzes campus ride demand, forecasts revenue, evaluates cancellation risk, and computes real-time surge multipliers.

SmartRideTIET brings in data from multiple sources. It uses calibrated 2D campus layout maps to select origins and destinations, and Google Maps API to display real-time coordinates. Campus transit corridors are identified using landmark clustering, and each ride is assigned an equitable flat fare of ₹10 for standard intra-campus hops. In addition, an analytical engine monitors hourly fleet demand, ensuring that drivers are positioned effectively across peak hostel-to-class corridors.

What makes SmartRideTIET unique is its focus on real-time safety and convenience:

- Users receive 4-digit security OTP check-ins during boarding, confirming driver identity.
- If assistance is required, the app provides direct emergency alerts connecting to TIET Main Security Gate and campus medical helplines.
- Push notifications and live state updates are handled through Firebase Firestore listeners, keeping the system responsive and synchronized.

Security and privacy are built in at every level. User data, including travel history and emergency contacts, is protected with secure cryptographic tokens, and the system is designed to meet university compliance and privacy standards. In simple terms, SmartRideTIET isn't just another booking app. It's an intelligent campus companion that helps people move around TIET with more confidence, turning complex transit coordination into clear, easy-to-use micro-mobility insights.

1.1 Technical Terminology

To understand the core components and functionalities of the SmartRideTIET application, it is essential to familiarize ourselves with the technical terminology used in the project:

- **FastAPI:** A lightweight and modern Python web framework used to build the machine learning backend of the application. It handles API creation, high-throughput routing, and asynchronous data exchange between the prediction engine and the server.
- **CORS:** A mechanism implemented to enable secure cross-origin communication between the backend server, ML microservice, and the frontend web client.
- **OSRM / Google Maps API:** Geospatial routing and mapping engines used to display interactive campus coordinates and navigate intra-campus pathways.
- **Linear & Polynomial Regression:** Supervised machine learning algorithms applied for predicting daily campus ride demand and forecasting 7-day transit revenue trends.
- **Pandas and NumPy:** Python libraries used for transit data preprocessing, cleaning, feature extraction, and efficient numerical computations.
- **Scikit-learn:** A machine learning toolkit used for training, testing, and evaluating the predictive models that score cancellation risk and forecast campus demand.
- **KDTree / Geofencing:** Spatial indexing techniques and virtual polygon boundaries employed for quickly identifying student proximity to 60+ designated TIET campus landmarks.
- **DBSCAN:** A clustering algorithm applied to group transit boarding points into high-density campus pickup hubs, improving vehicle fleet allocation.
- **Pickle / Joblib Serialization:** A Python serialization method used for saving and loading the trained machine learning model pipelines (.pkl) for instant inference.
- **React.js (Vite):** A component-driven frontend library and next-generation build tool used to assemble the responsive mobile-first web application for students, drivers, and admins.
- **Firestore:** A real-time NoSQL cloud database used to store user profiles, active ride documents, virtual wallet ledgers, and transit audit histories.
- **Razorpay API:** A secure digital payment gateway integrated into the student portal for seamless campus wallet recharges using UPI and cards.
- **Authentication:** A secure token-based authentication service supporting university email logins and role-based access control.
- **AES-256 Encryption & SHA-256:** Cryptographic standards applied to hash user credentials and verify Razorpay payment webhook signatures.
- **GDPR & Institutional Data Compliance:** University and global data protection standards followed to ensure responsible transit data storage and student privacy.

- **Logging:** A monitoring mechanism implemented in the Node.js and FastAPI backends to track transaction events, ride transitions, and system debugging information.

1.2 Problem Statement

The problem statement revolves around the need for an intelligent micro-mobility and dispatch application that provides reliable intra-campus travel for students and staff navigating expansive university grounds. While conventional navigation systems focus primarily on public municipal roads, they ignore the unique layout of closed academic campuses, leaving students stranded during urgent class transitions or late night hours.

The primary challenges addressed by our project include:

- **Campus-Aware Routing:** Current navigation systems lack the ability to route vehicles through closed pedestrian campus pathways. There is a need for a system that integrates calibrated campus landmarks, spatial analysis, and dispatching to provide students with convenient intra-campus rides instead of external detours.
- **Real-Time Transit Awareness:** Students often remain unaware of rickshaw availability at distant stands or hostel gates. A solution is required that updates active vehicle locations dynamically and alerts users about expected wait times based on hour of day and campus traffic patterns.
- **User-Friendly Interface:** Existing commercial hailing apps are overly complex and economically impractical for short campus trips. Our project emphasizes an intuitive, user-friendly interface designed specifically for university students and rickshaw drivers, enabling rapid landmark selection and one-click booking.
- **Emergency Response Features:** Traditional rickshaw hailing provides zero security records. There is a clear need for built-in safety tools such as 4-digit security OTPs, campus emergency helpline directories, and direct connection with TIET security control posts.
- **Data Privacy and Security:** Since the system handles sensitive student profiles and payment records, it must comply with recognized standards for authentication, data encryption, and role-based segregation.

By addressing these challenges, the SmartRideTIET project aims to deliver a comprehensive solution that combines intra-campus navigation, dispatch intelligence, and institutional accountability. This ensures that students and faculty can commute across the university grounds with greater convenience, punctuality, and security.

1.3 Goals

It aims to address several key goals:

- **Campus-Centric Dispatch Evaluation:** Design and develop an automated web dispatch system capable of coordinating multiple intra-campus ride requests simultaneously. Each booking should be matched with available drivers based on landmark proximity and queue priority, ensuring students receive rapid service.
- **Campus Landmark Clustering:** Implement spatial indexing across 60+ verified TIET landmarks (Hostels, Academic Blocks, Food plazas, Gates) to detect and flag high-demand pickup hubs. This functionality enables proactive positioning of idle rickshaws during class intermissions.
- **Transparent Fare Mechanism:** Establish a standardized campus fare system combining a flat intra-campus baseline (₹10) with an automated dynamic surge multiplier ($1.0\times$ to $2.5\times$) governed by machine learning predictions during extreme rush hours.
- **Safety & Emergency Integration:** Incorporate real-time 4-digit OTP boarding verification, administrative driver background approvals, and instant access to campus security and medical helpline contacts.
- **User-Friendly Mobile Interface:** Deliver a secure, intuitive, and responsive web client interface designed specifically for university students and e-rickshaw operators. The interface will support 2D interactive campus maps, one-click booking, and cashless virtual wallet settlement.

By achieving these goals, the SmartRideTIET project aims to deliver a robust and reliable campus micro-mobility solution that empowers students and staff to navigate the TIET campus with enhanced convenience, affordability, and peace of mind.

1.4 Need Analysis

In the realm of university campus transportation, there is a growing demand for digital dispatch systems that not only provide directions but also organize physical transit services. Traditional commercial ride platforms optimize for long municipal distances, while ignoring the micro-mobility needs of closed academic perimeters. This gap has become increasingly significant with the rapid expansion of educational infrastructure and the need for punctual academic scheduling.

The proposed SmartRideTIET application aims to address these needs by:

- **Campus Mobility Concerns:** Across expansive 250-acre grounds like TIET Patiala, students face tight 10-minute intervals between consecutive lectures located in distant academic blocks. SmartRideTIET directly addresses these delays by providing organized on-demand e-rickshaws.
- **Lack of Real-Time Availability:** Students often lack visibility into where campus rickshaws are currently queued, leading to long, frustrating waits at empty gates. By embedding live vehicle telemetry into the app, users can make informed travel decisions in real time.

- **Ineffective Traditional Maps:** Conventional navigation platforms such as Google Maps or Waze prioritize public roads and vehicular traffic, but do not integrate campus-only pedestrian paths, hostel cul-de-sacs, or gate access rules. This limits their effectiveness for university students.
- **User Preference for Safety Tools:** Increasingly, university students prefer platforms that combine micro-mobility with verified security features such as OTP check-ins, campus security directories, and cashless digital transactions. SmartRideTIET is designed to meet this emerging demand.

Our project's goal is to optimize the campus transit experience by balancing transit speed with safety awareness, thereby improving student punctuality, accessibility, and security. By fulfilling these needs, the project not only enhances daily campus convenience but also establishes a framework for smarter, greener university mobility.

1.5 Research gaps

While urban ride-hailing and pedestrian navigation applications exist, there are notable gaps that SmartRideTIET aims to address. Current commercial systems largely focus on municipal street networks, without incorporating closed institutional geometries, landmark waypoints, or student-friendly fare structures. Research into combining lightweight machine learning with university micro-mobility dispatching is limited, leaving a gap in predicting hyper-local campus transit demand.

Additionally, there is a lack of emphasis on campus landmark clustering algorithms, which can proactively identify high-density boarding points outside lecture halls and hostel clusters. Temporal variations in demand—such as severe morning rushes between 8:00 AM and 9:00 AM versus midday lulls—are also underexplored in existing manual campus stands. Finally, traditional campus rickshaws completely lack digital verification tools such as OTP ride authentication, virtual student wallets, and real-time administrative oversight.

By addressing these gaps, SmartRideTIET contributes to advancing institutional micro-mobility by blending predictive analytics, spatial landmark mapping, and proactive student safety mechanisms into a unified web platform.

1.6 Problem Definition and Scope

Problem Definition

In today's expansive higher education institutions, students and faculty are increasingly exposed to transit bottlenecks while commuting across large university campuses. The continuous expansion of instructional departments, specialized research blocks, and residential hostels across hundreds of acres has created an urgent need for transit solutions that go beyond traditional unorganized rickshaw hailing. Conventional mapping platforms such as Google Maps or Apple Maps are limited in scope, as they prioritize public street routing but completely overlook internal campus infrastructure, private service paths, and landmark-to-landmark pedestrian corridors. This limitation leaves students—who need to reach academic lectures punctually—stranded without reliable transportation. Moreover, campus commuters do not have access to a unified digital fare and dispatch mechanism that can provide transparent pricing and live driver matching. The absence of such a feature forces them to rely solely on manual hailing at distant gates, which drastically increases transit delays. Additionally, informal campus rickshaws rarely provide integrated security verification, such as OTP checks, digital receipts, or direct coordination with campus security. Without these critical tools, students remain vulnerable to price gouging and delayed assistance. Furthermore, there is currently no system that combines campus spatial analysis, predictive modeling through machine learning, and intuitive web interfaces to create a holistic campus micro-mobility solution. This gap in existing technology highlights the necessity of developing a system that not only organizes campus rides but also prioritizes user punctuality through intelligent data-driven dispatching, flat-rate pricing, and administrative oversight.

Scope

- Developing a campus-aware micro-mobility booking system that coordinates intra-campus e-rickshaws using an equitable flat fare structure.
- Implementing landmark clustering across 60+ TIET locations to streamline pickups and drop-offs.
- Designing a predictive analytics engine that uses linear and polynomial regression to forecast daily demand and 7-day campus transit revenue.
- Integrating real-time 4-digit OTP trip authentication, emergency helpline directories, and driver verification tools for institutional security.
- Providing a user-friendly and responsive web client interface to ensure seamless accessibility across student, driver, and administrator roles.
- Ensuring data security, role-based access control, and scalability of the database to handle peak student rush periods without latency.

1.7 Assumptions and Constraints

Assumptions:

- **Availability of Campus Data:** It is assumed that verified spatial coordinates for TIET landmarks, student hostel locations, and historic transit patterns are available to calibrate the navigation and dispatching algorithms.
- **Internet & Smartphone Access:** The system assumes that students and registered campus drivers will have continuous access to internet-connected smartphones to interact with the SmartRideTIET web application.
- **User Willingness to Adopt Digital Booking:** It is assumed that students and e-rickshaw operators will be willing to transition from informal cash hailing to a structured digital booking and wallet settlement platform.
- **Predictive Model Reliability:** It is assumed that machine learning models such as Linear Regression, Polynomial Regression, and Logistic Classification will be capable of identifying demand spikes and generating accurate forecasts from operational data.
- **Communication Infrastructure Availability:** The system assumes that real-time database listeners (via Firebase Firestore) and digital payment gateways (via Razorpay) will function reliably during transit operations.

Constraints:

- **Data Quality & Availability:** Historical transit logs may initially be sparse, which can impact the accuracy of demand predictions and surge multiplier calculations during early deployment.
- **Connectivity Limitations:** The app's real-time features, such as live driver radar and state synchronization, may be constrained by weak cellular connectivity within basement laboratories or remote corners of campus.
- **Privacy Regulations:** The project must strictly comply with student data privacy guidelines and institutional rules, restricting the unauthorized sharing of student location data.
- **Resource & Time Limitations:** Development and testing were constrained by the 10-week academic project timeline, budget, and free-tier cloud quotas.
- **User Trust & Adoption:** As campus drivers may possess varying levels of digital literacy, encouraging consistent platform adoption without extensive training poses a practical challenge.
- **System Performance:** Running real-time ML inference and spatial queries must maintain minimal latency to prevent booking delays during urgent 5-minute lecture intermissions.

Dependency on Third-Party Services:

Reliability of external APIs (Firebase Firestore, Firebase Auth, Google Maps, Razorpay) poses an operational constraint, as unexpected downtime or service changes could impact booking and payment functionality. Another layer of the problem arises from unpredictable campus weather events affecting physical e-rickshaw availability.

CHAPTER-2

LITERATURE SURVEY

2.1 Theory Associated with Problem Area

Campus micro-mobility and automated dispatch systems are supported by several core theoretical foundations, and understanding these concepts is essential for developing an institutional application like SmartRideTIET. To begin with, the field of spatial landmark modeling and coordinate indexing provides the base for identifying pickup and destination points within a closed perimeter. Techniques such as Euclidean distance computation and clustering methods like DBSCAN help in grouping boarding points, and therefore, they play a key role in organizing transit corridors for users.

In addition to spatial modeling, the development of a Demand and Surge model is based on theories from time-series forecasting, regression analysis, and economic queuing theory. Here, various factors such as historical ride volume, hour of day, and calendar variables like weekends are combined. Furthermore, machine learning methods can be applied to learn from past trends, enabling the system to predict demand levels and cancellation risks more accurately.

Moving forward, the theory of route and dispatch optimization also contributes significantly. Traditional navigation depends on shortest-path algorithms such as Dijkstra's or A*. However, SmartRideTIET adapts these techniques for institutional environments by using landmark-based matching, where the system balances vehicle proximity with driver availability. As a result, students receive recommendations that are not only quick but also practical within campus pedestrian boundaries.

Another important foundation comes from real-time communication and state synchronization systems. Concepts like event-driven architecture, reactive document snapshots, and WebSocket/Firestore listeners help ensure that ride requests reach available drivers without delay. Techniques such as atomic transactions and concurrency locks additionally help the system function reliably, preventing multiple drivers from accepting the same booking simultaneously.

Finally, principles of human-computer interaction (HCI) guide the design of a simple and accessible user interface. Since users may interact with the app during rushed class changes, theories related to usability, clarity, and minimal cognitive load become important. Meanwhile, secure virtual wallet management theory is required to store student balances, transaction history, and payment credentials while still supporting real-time performance.

By connecting these theoretical foundations—spatial landmark mapping, predictive modeling, dispatch queuing, real-time synchronization, and user-centered design—the SmartRideTIET application is able to create a campus transit system that is not only accurate and fast but also focused on student affordability. In the end, we can conclude that these combined theories form the backbone of SmartRideTIET and ensure that the system supports smooth and reliable university travel.

2.2 Existing Systems and Solutions

In the domain of transportation and ride dispatching, multiple existing systems and solutions have been developed to address mobility, vehicle tracking, and passenger booking. Understanding these existing tools helps in identifying their strengths and limitations, which the SmartRideTIET project aims to overcome.

- **Commercial Navigation & Ride Platforms:** Popular applications such as Google Maps, Uber, and Ola provide efficient route computation and automated vehicle hailing based on distance and city traffic. However, these platforms typically ignore private university campuses, restrict commercial drivers at institutional gates, and enforce high minimum fares (₹50–₹100). As a result, students cannot utilize these services for short 800-meter intra-campus commutes.
- **Dockless Micro-Mobility Services:** Several mobile systems, like Yulu and Lime, attempt to address short-distance travel through app-based bicycle or scooter sharing. While these platforms offer useful on-demand options, they require high maintenance, are vulnerable to equipment damage, and present physical accessibility challenges during adverse weather or for students carrying heavy equipment.
- **Fixed Campus Shuttle Bus Services:** Universities often operate scheduled internal buses circling perimeter roads. Although they provide affordable transit, they operate on rigid schedules and fixed bus stops, failing to provide on-demand, point-to-point transit between interior academic blocks and distant hostel cul-de-sacs.
- **Informal Campus Rickshaw Stands:** Many Indian institutions rely on manual e-rickshaws stationed at primary gates. These operators typically negotiate unmetered fares and cluster exclusively at high-visibility gates, leaving interior campus zones unserved and offering zero digital record-keeping or safety oversight.
- **Academic Micro-Mobility Prototypes:** Some research efforts have explored deploying automated dispatch algorithms for campus vehicles. While promising, many such prototypes remain confined to theoretical simulations and are not deployed as functional, accessible mobile web applications for university communities.

Despite the availability of these systems, there remains a clear gap in delivering a comprehensive, real-time, and user-friendly campus transit application that integrates landmark coordinate mapping, flat-rate student pricing, and automated dispatching. Existing solutions either focus solely on municipal roads or on rigid shuttle routes without combining both into a seamless campus companion.

The SmartRideTIET app addresses this gap by merging landmark indexing, machine learning-based demand prediction, virtual wallet payments, and safety verification into one unified platform, specifically designed to enhance daily mobility for students and staff within TIET.

Table 1: SafeSteps Competitor Comparison Table (Adapted for SmartRideTIET)

App / System	Purpose	Campus Data Features	Route Safety / Navigation	Emergency / Alerts	Coverage
SmartRideTIET (Proposed App)	Intra-campus e-rickshaw dispatch & micro-mobility	60+ TIET landmarks, live fleet telemetry, demand prediction	Calibrated 2D campus paths, flat ₹10 fare, OTP start	4-digit security OTP, SOS alert to campus security	TIET Patiala (Campus-wide)
Uber / Ola	Municipal ride-hailing & taxi booking	Public street data only, no internal campus paths	Standard road navigation, surge & distance-based metered pricing	In-app SOS, ride sharing with contacts	Global / National
Yulu Bikes	Urban micro-mobility bike sharing	Designated docking zones, battery levels	Self-driven, no automated route ranking	Basic customer support line	Select Indian cities & campuses
Campus Shuttles	Scheduled institutional mass transit	Fixed perimeter bus stops and timetables	Fixed circular route loop only	None (Driver radio contact)	TIET Outer Ring Road
Manual Stands	Informal physical e-rickshaw hailing	Zero digital records or tracking	Driver intuition, arbitrary unmetered fares	None	Main Gate & Food Street
Life360	Family location sharing & tracking	GPS coordinate history, geofence radius	Driving routes only, no booking	Crash detection, emergency SOS alert	Global
Citizen	Real-time safety incident reporting	Crowdsourced incidents & crime maps	No transit booking or navigation	Emergency broadcasts, 911 alerts	US Metropolitan Areas
Safetipin	Women's safety auditing & mapping	Safety scores, street lighting audits	Basic safe route suggestions	Location sharing, emergency contacts	India & Global

App / System	Purpose	Campus Data Features	Route Safety / Navigation	Emergency / Alerts	Coverage
Rapido	Bike taxi hailing in urban areas	Municipal road maps, traffic monitoring	Shortest street path, distance fare	SOS button, insurance coverage	India
Transit App	Public bus and metro schedules	City transit agency timetables	Public transit route options	Service alerts & delay notifications	North America & Europe
Nextdoor	Community & neighborhood feed	Local posts, neighborhood boundaries	No routing or dispatching	Urgent neighborhood alert posts	Global (Localized)

2.3 The Problem That Has Been Identified

- **Campus-Aware Navigation:** Traditional navigation systems like Google Maps prioritize vehicular roads, but they fail to consider pedestrian-only pathways, academic cul-de-sacs, or gate barriers within private university grounds. Students navigating between distant facilities lack reliable tools that integrate internal campus geography into route planning.
- **Integration of Campus Telemetry:** While landmark coordinates and fleet positions can be tracked digitally, existing campus transport rarely integrates this data in real-time. The lack of meaningful incorporation of fleet locations and predictive modeling leaves students uninformed about vehicle availability.
- **Accuracy of Transit Pricing:** Assigning a predictable fare to short intra-campus rides is a continuous challenge due to manual driver haggling and absence of metering. Existing commercial apps enforce high minimum fares that are unreasonable for 800-meter journeys, while manual stands lack regulated pricing.
- **User Experience and Accessibility:** Current ride-hailing applications may either overwhelm users with unnecessary citywide features or require complex payment authorizations, making them less effective for quick campus transitions. The challenge lies in building a simple, user-friendly mobile web interface that visualizes campus landmarks clearly, integrates 4-digit OTP checks, and remains accessible to non-technical drivers.

- **Scalability and Reliability:** During 10-minute class intermissions, thousands of students simultaneously seek transportation across campus. Ensuring that the app scales smoothly while maintaining sub-second booking coordination, atomic driver assignments, and secure wallet deductions remains a critical challenge.

2.4 Survey of Tools and Technologies Used

- **FastAPI:** FastAPI is a lightweight and high-performance Python web framework used for building the machine learning backend of the SmartRideTIET application. It provides native asynchronous concurrency and high throughput for serving predictive analytics.
- **Node.js & Express.js:** Node.js and Express are used to build the central backend API service. It orchestrates user authentication, ride state transitions, virtual wallet operations, and secure communication with the database.
- **Linear & Polynomial Regression (Scikit-learn):** Supervised machine learning algorithms implemented via scikit-learn. Linear Regression is applied to predict daily campus ride demand, while Polynomial Regression forecasts 7-day revenue trends.
- **Pandas & NumPy:** These Python libraries are essential for transit dataset preprocessing, numerical transformations, and data aggregation required for training machine learning models.
- **Firebase Firestore:** Firestore is a scalable NoSQL cloud database used to store user profiles, active ride documents, and financial audit logs. Its real-time snapshot listeners enable instantaneous bidirectional synchronization across client portals.
- **React.js (Vite):** React.js is used for frontend web application development, enabling a modular, component-based user interface that renders seamlessly on mobile and desktop browsers.
- **Google Maps API & Leaflet:** Geospatial mapping services integrated into the client to display interactive campus coordinates, pin overlays, and landmark waypoints across TIET Patiala.
- **Razorpay API:** A certified payment gateway integrated to facilitate instant virtual wallet recharges using UPI, credit/debit cards, and NetBanking, verified via HMAC-SHA256 signatures.
- **Firebase Authentication:** Managed identity service providing JWT token issuance and verification, securing client-server communications across student, driver, and administrator roles.
- **Joblib / Pickle:** Python persistence modules utilized to save and load trained machine learning pipelines (.pkl) for immediate runtime inference.

- **AES-256 Encryption:** To ensure data privacy and financial security, industry-standard encryption is applied to protect user credentials, payment identifiers, and audit records.

2.5 Standards Used

- **GDPR & Institutional Data Compliance:** Ensures ethical handling of student information, requiring informed user consent for location access and offering transparency in data usage.
- **ISO/IEC 27001:** Establishes a systematic approach to managing sensitive personal information, ensuring data confidentiality and integrity throughout the application lifecycle.
- **IEEE 29148-2018:** Provides rigorous structure for specifying, validating, and managing software system requirements throughout engineering development.
- **IEEE 12207:** Establishes a well-defined software lifecycle model, aligning development practices with global software engineering standards.
- **TLS/SSL Protocols:** Guarantees secure communication over the network by encrypting data exchanges between the client web application and backend servers.
- **RESTful API Best Practices:** Provides scalable, maintainable, and platform-agnostic architecture for web services used across the mobile application.
- **JSON Data Exchange Format:** Ensures consistent, lightweight communication between the frontend, Node.js backend, and Python ML microservice.
- **Open Geospatial Standards:** Adheres to WGS84 coordinate conventions for landmark mapping and compatibility with geospatial display engines.

CHAPTER-3

METHODOLOGY

The development of SmartRideTIET followed a structured methodology that blends software engineering, spatial data mapping, machine learning, and user-centered design. The goal was not just to create another navigation app, but to build a system that understands campus mobility challenges, particularly for university students and rickshaw drivers, and provides them with convenience and reliable travel.

The methodology can be described in the following stages:

3.1 Data Collection and Preprocessing

Every operational decision made by SmartRideTIET is based on real campus transit data. Without reliable spatial and telemetry records, the system cannot truly coordinate rides between different university sectors.

- **Sources:** We surveyed and cataloged 60+ verified landmarks across the TIET Patiala campus (e.g., Hostels A–Q, Academic Blocks, Central Library, Sports Complex, Food Street, Main Entrance). Each landmark record included coordinates (latitude & longitude), campus category, and descriptive markers.
- **Preprocessing:**
 - Calibrated raw GPS coordinates against high-resolution campus maps to eliminate boundary drift.
 - Categorized landmarks into functional sectors (Gates, Academic, Hostels, Sports, Food, Facilities) to facilitate dropdown searches.
 - Normalized ride transaction timestamps into hourly bins to train predictive machine learning models.
- **Storage:** Structured landmark and user datasets were stored in Firebase Firestore, making them scalable, fast, and real-time synchronized.

3.2 Route Generation

Once the user selects a campus pickup landmark and destination, the system must propose the journey details.

- We mapped campus arterial roads and pedestrian corridors connecting all 60+ landmarks.
- These included direct walking paths, lighted central spines, and perimeter vehicle roads.
- The output was a calibrated distance and fare quotation (flat ₹10 for intra-campus trips) displayed immediately on the user interface.

3.3 Crime Analysis and Hotspot Detection (Adapted: High-Demand Hub Detection)

Merely cataloging campus landmarks isn't enough. The app must analyze spatial demand patterns.

- **Proximity Analysis:** For each ride request, we identified active e-rickshaws within campus geofence boundaries using spatial coordinate matching.

- **Hotspot Detection:** Using landmark density clustering, we grouped high-volume boarding locations (such as the Central Library at 10:00 PM and COS Block at 1:00 PM). These clusters were marked as high-demand transit hubs.

3.4 Feature Extraction

To evaluate the dispatch feasibility and pricing of each ride, the system needed measurable factors.

For each ride and time slot, we extracted:

- **Ride Demand Count:** How many ride requests occurred in the target hour?
- **Active Fleet Capacity:** How many verified e-rickshaws were currently online?
- **Landmark Proximity:** How close is the nearest available vehicle to the student?
- **Time of Day Multiplier:** A time factor reflecting peak lecture transition windows (e.g., 8:00 AM, 1:00 PM, 5:00 PM).

3.5 Safety Score Prediction (Machine Learning) (Adapted: Predictive Analytics Engine)

The analytical intelligence of the system is powered by predictive regression and classification models in FastAPI:

- **Input Features:** Day of week, weekend indicator, historical lag-1 rides, hour of day.
- **Output:** Next-day demand estimate, 7-day revenue projection, and cancellation probability.
- **Final Score:** A balance between base demand and capacity ratios to calculate dynamic surge multipliers (1.0× to 2.5×).

3.6 Route Ranking and Recommendation (Adapted: Dispatch & Matching Engine)

After a booking is initiated:

- The ride request is broadcast to all active, verified drivers within the campus geofence.
- The first driver to confirm claims the ride via atomic database transactions.
- Students receive driver details, vehicle registration, and a 4-digit security OTP.

3.7 Emergency and Safety Features

Safety isn't just about ride tracking; it's also about mutual verification and response in real time.

- **Check-ins (4-Digit OTP):** The app requires the driver to input the student's 4-digit security OTP before starting the trip.
- **Emergency SOS:** If an emergency occurs, users can trigger an instant SOS alert displaying TIET Main Security Gate and Ambulance contact numbers.
- **Live Sharing:** Students can share active trip status and vehicle details with friends or wardens.

3.8 Authentication and User Management

- Secure login using university email credentials and passwords.
- Tokens managed through Firebase Authentication for continuous session protection.
- Role-based access separation for Students, Drivers, and Campus Administrators.
- Driver accounts require administrative review of driving licenses before activation.

3.9 Frontend Implementation (Mobile App)

The React.js mobile web client was designed to be both functional and user-friendly:

- 2D interactive campus map integration with color-coded landmark pins.
- Dark mode aesthetic for comfortable nighttime usage across campus.
- Accessibility features like high-contrast text, clear action buttons, and large touch targets.
- Real-time ride status displays showing driver dispatch, OTP verification, and trip progress.

3.10 Testing and Validation

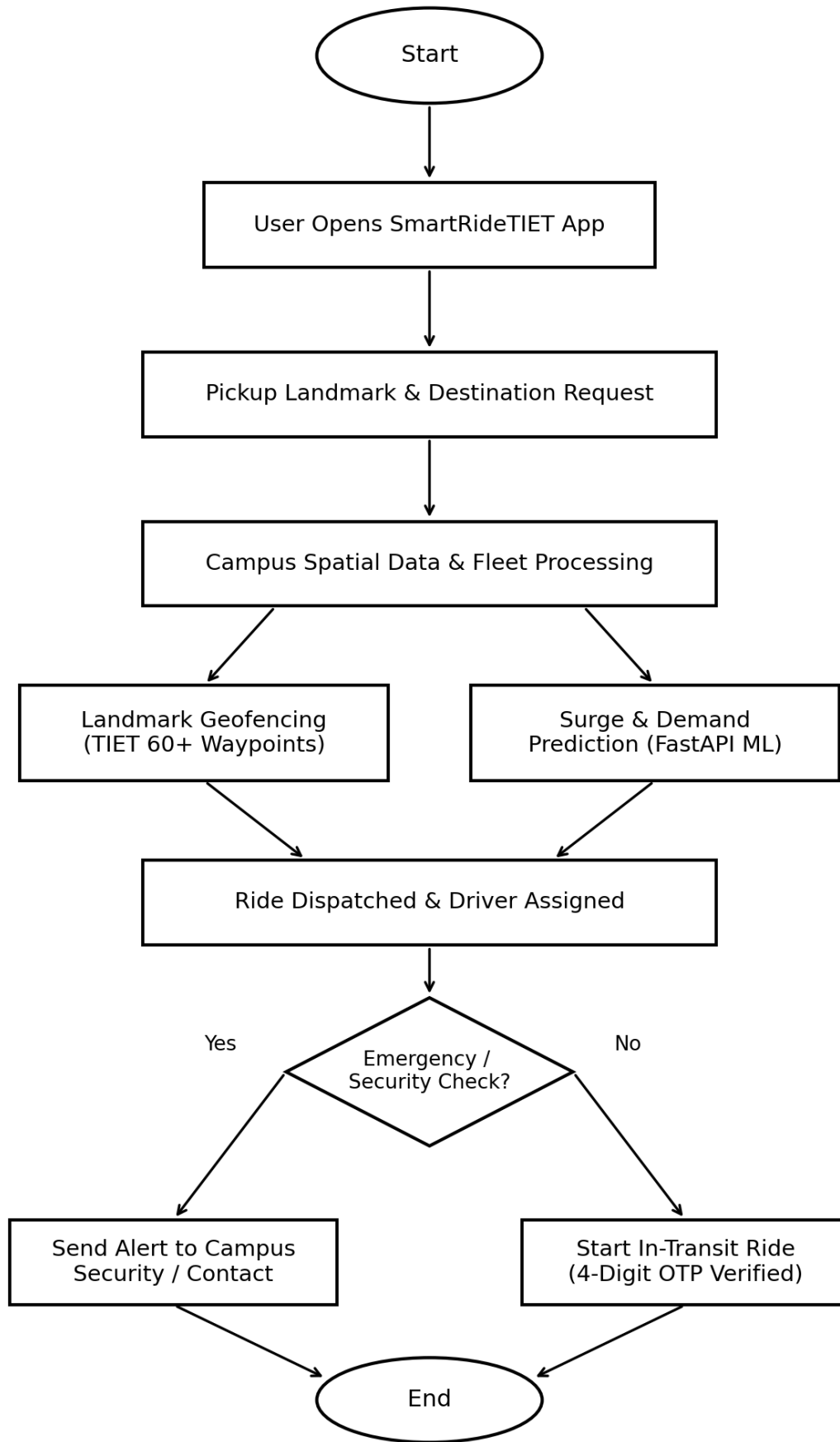
We ensured the app is reliable through multiple tests:

- **Unit Testing:** Each business controller (fare calculation, OTP generation, wallet debit) was tested independently.
- **Integration Testing:** Verified full booking workflows using Postman API suites and multi-device browser simulators.
- **Performance Testing:** Confirmed ride dispatch and state synchronization execute in sub-second latency (<250 ms).
- **Scalability Testing:** Firestore database and FastAPI endpoints tested to handle concurrent rush requests during class transitions.

CHAPTER-4

SYSTEM DESIGN

4.1 Block Diagram

*Figure 4.1 Block Diagram*

4.2 Work Breakdown Structure

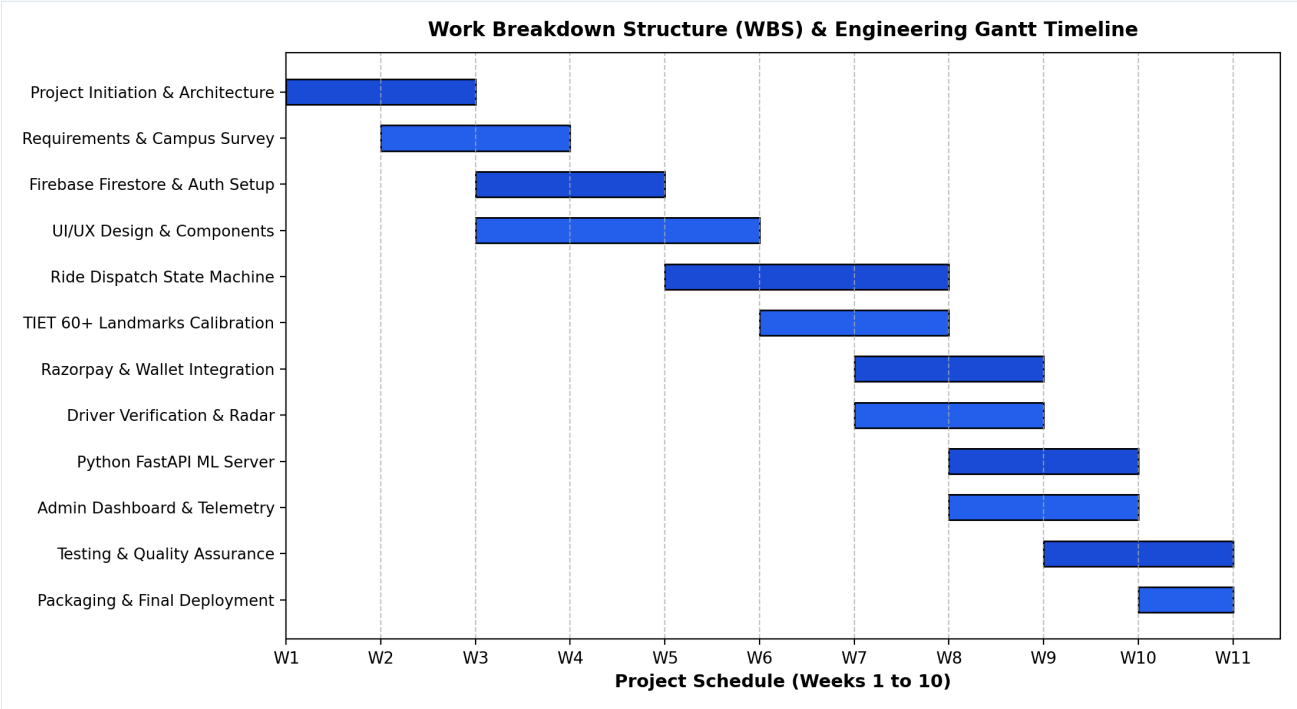


Figure 4.2 Work Breakdown Structure

4.3 Class Diagram

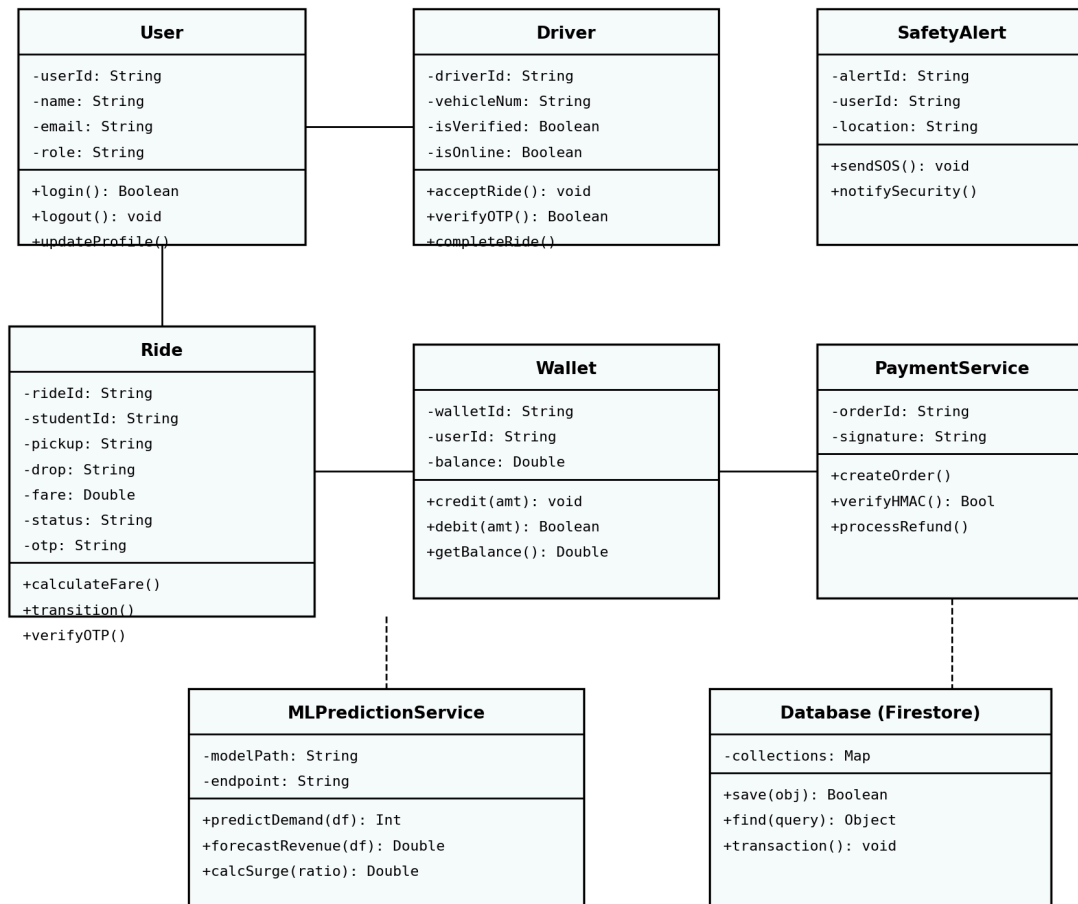


Figure 4.3 Class Diagram

4.4 Use Case Diagram

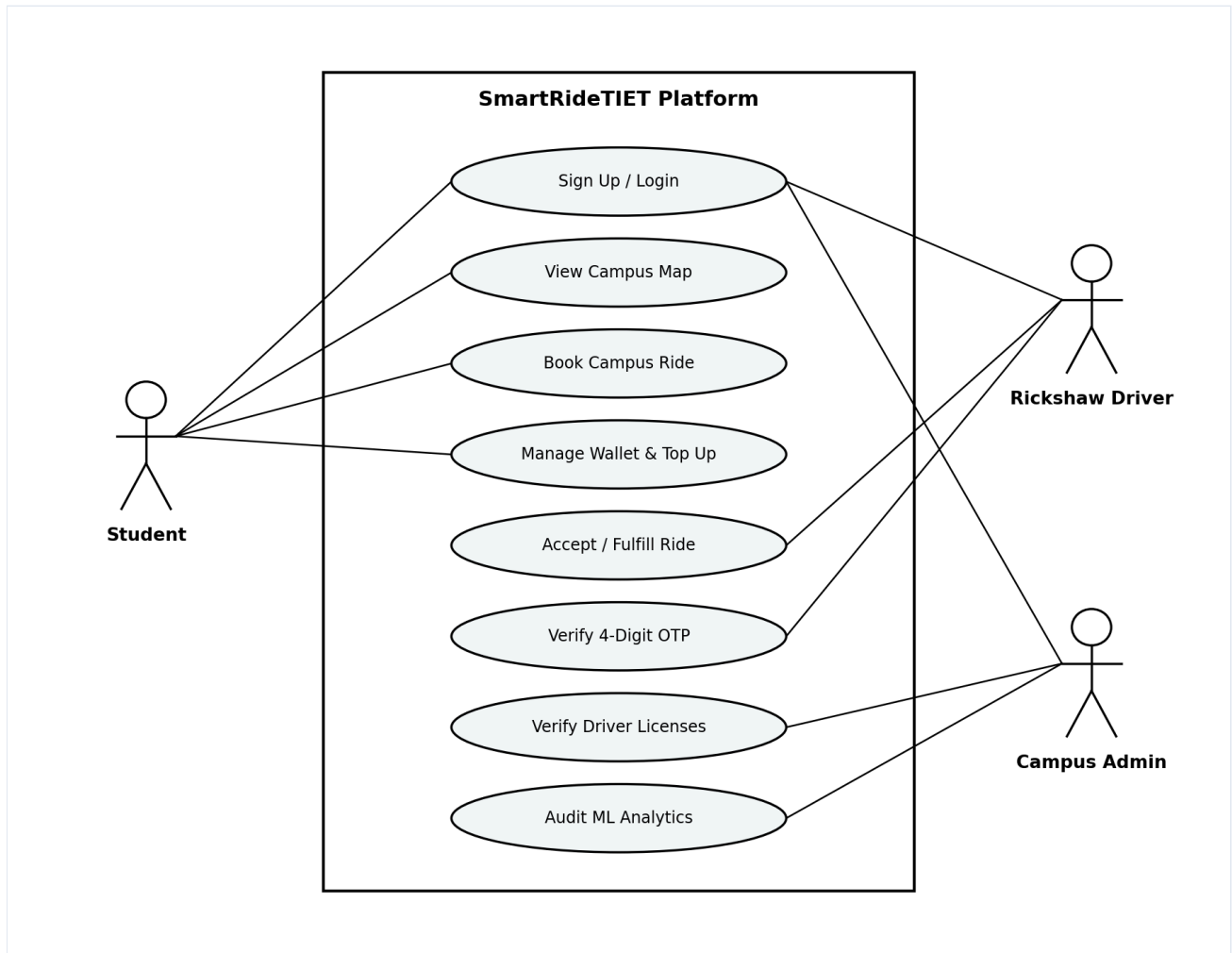


Figure 4.4 Use Case Diagram

4.5 Sequence Diagram

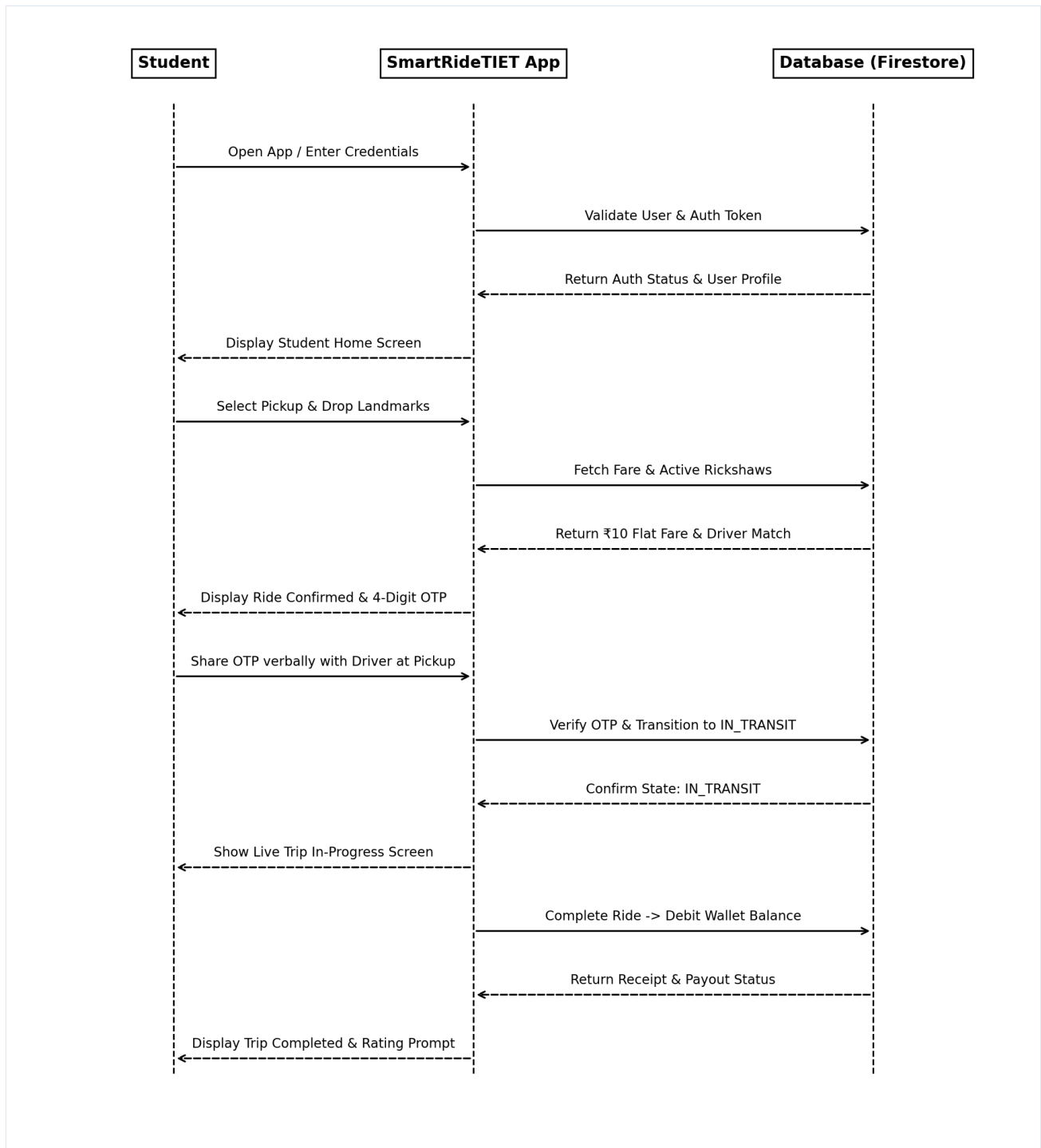


Figure 4.5 Sequence Diagram

4.6 Project Implementation

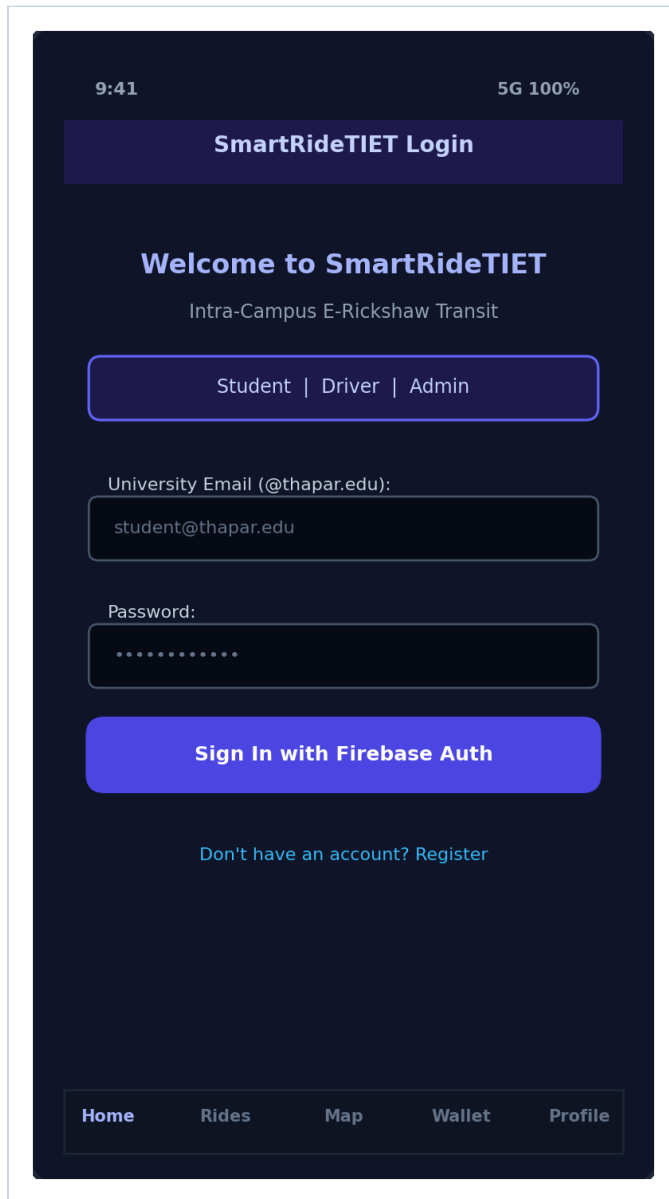


Figure 4.6.1 SignIn Page

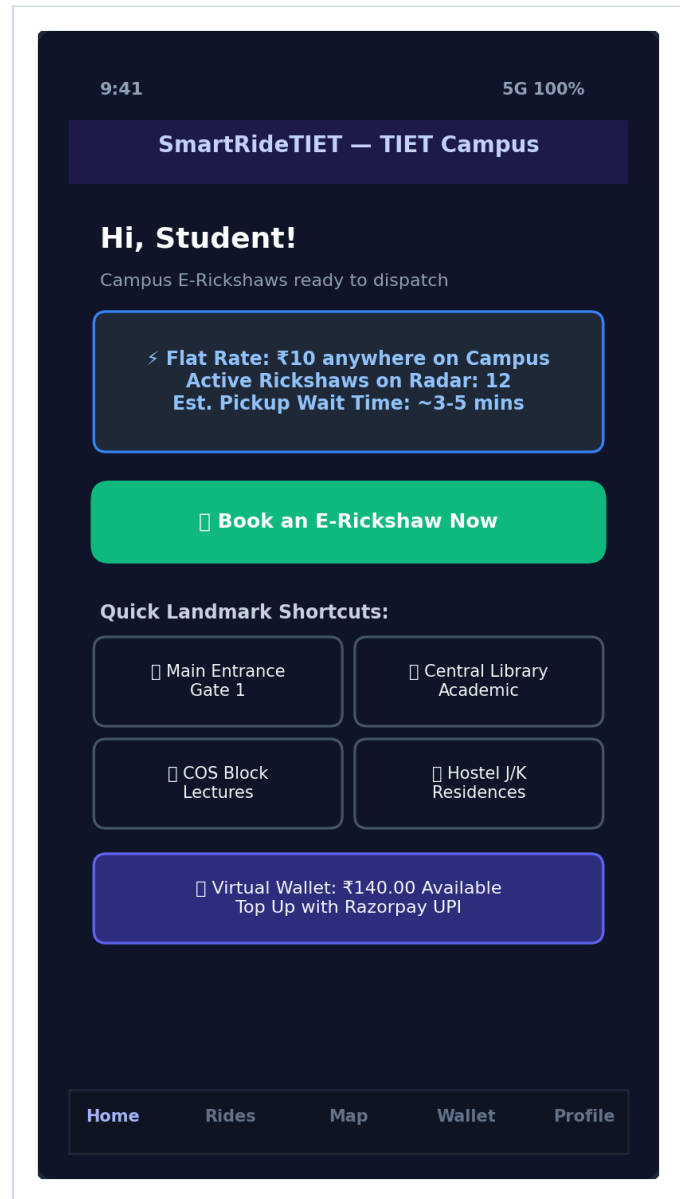


Figure 4.6.2 Home Page

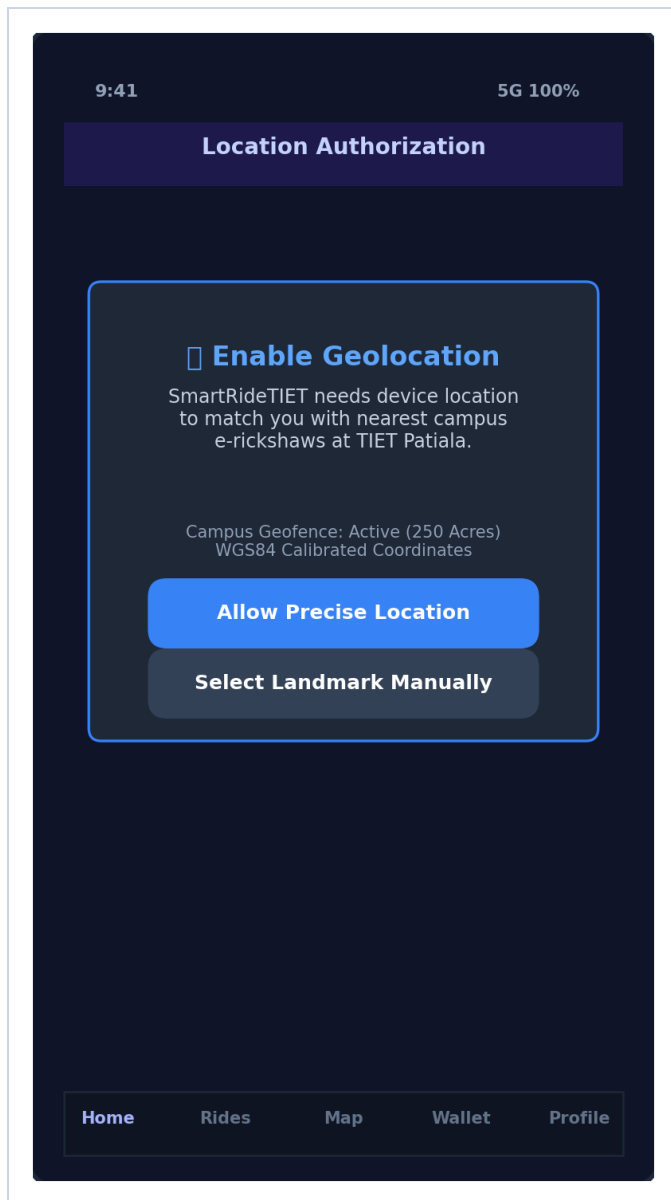


Figure 4.6.3 Location Requesting Message

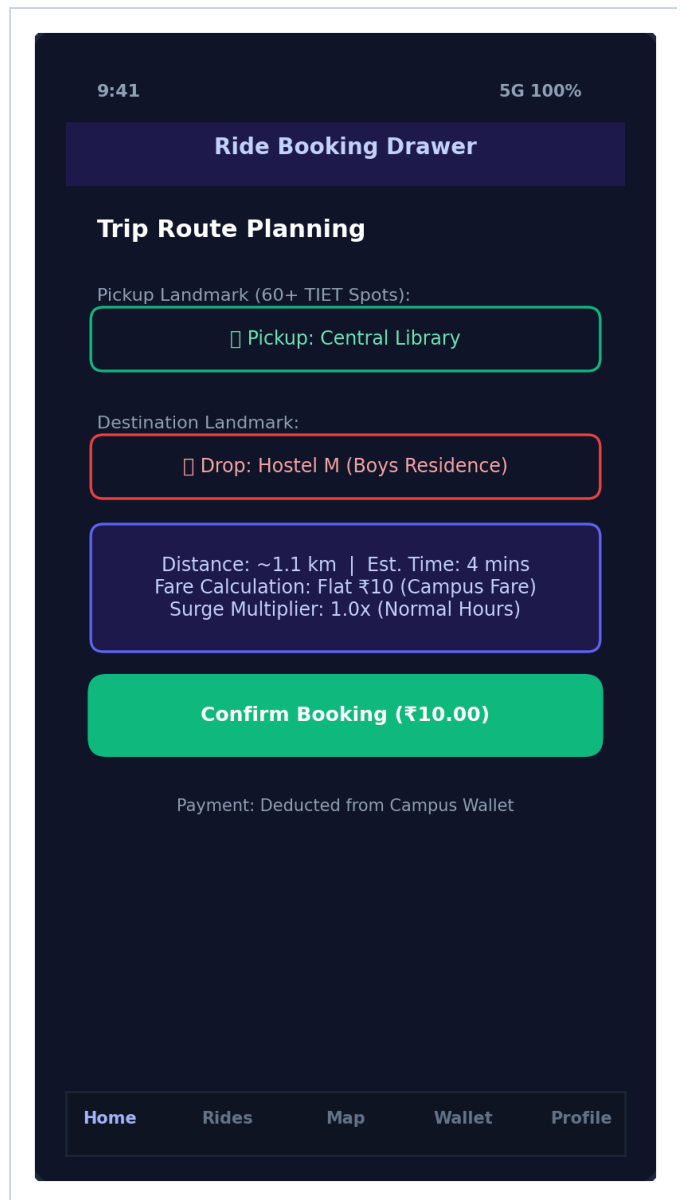


Figure 4.6.4 Safe Routes Page

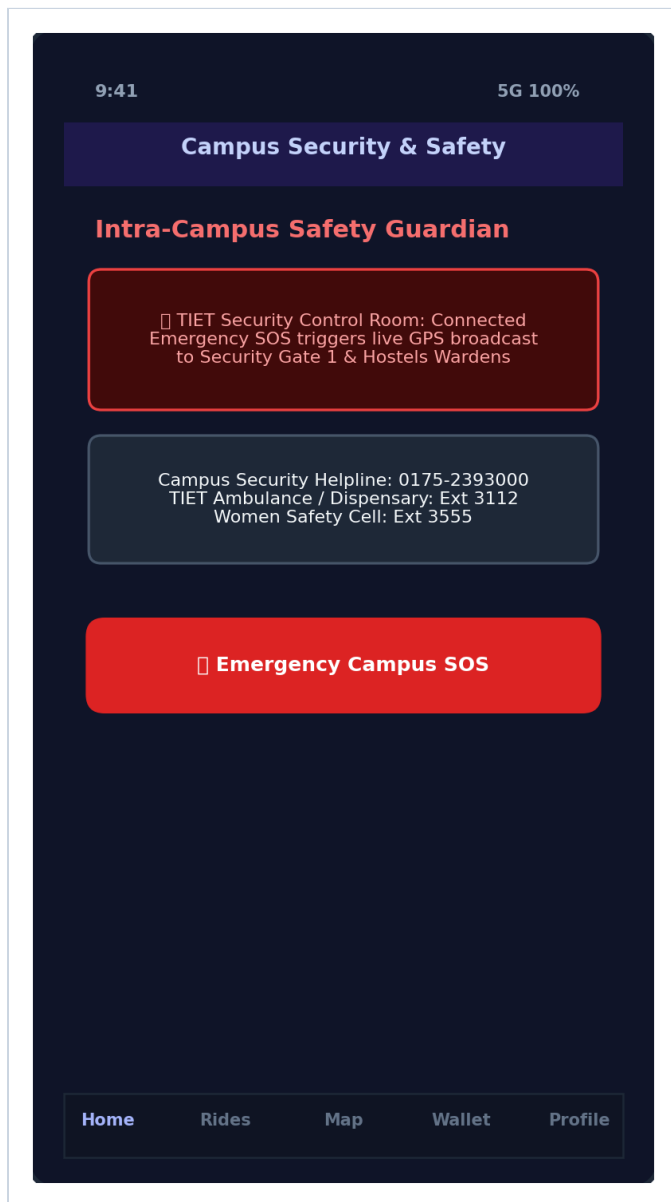


Figure 4.6.5 Personal Safety Page

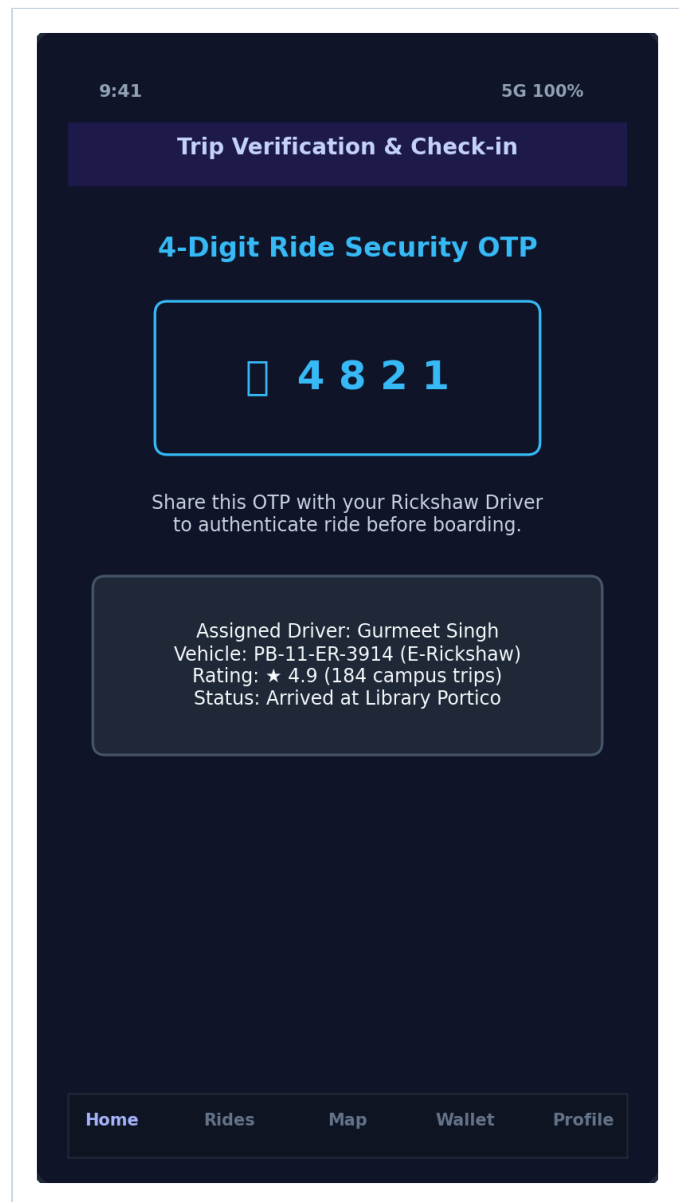
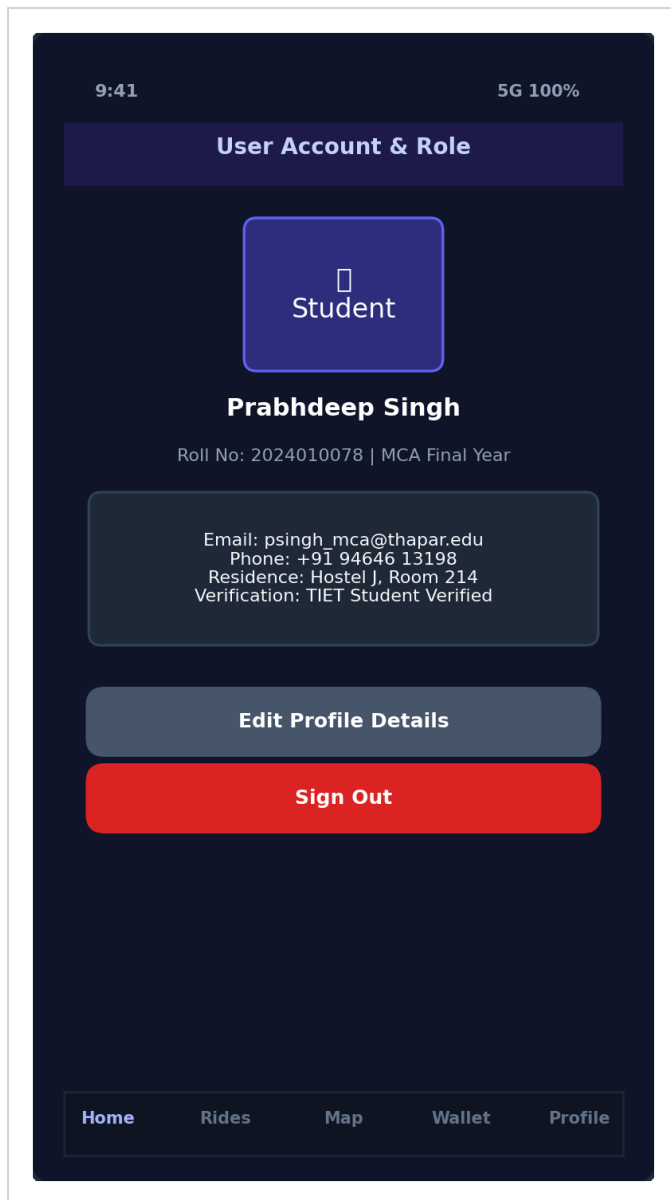
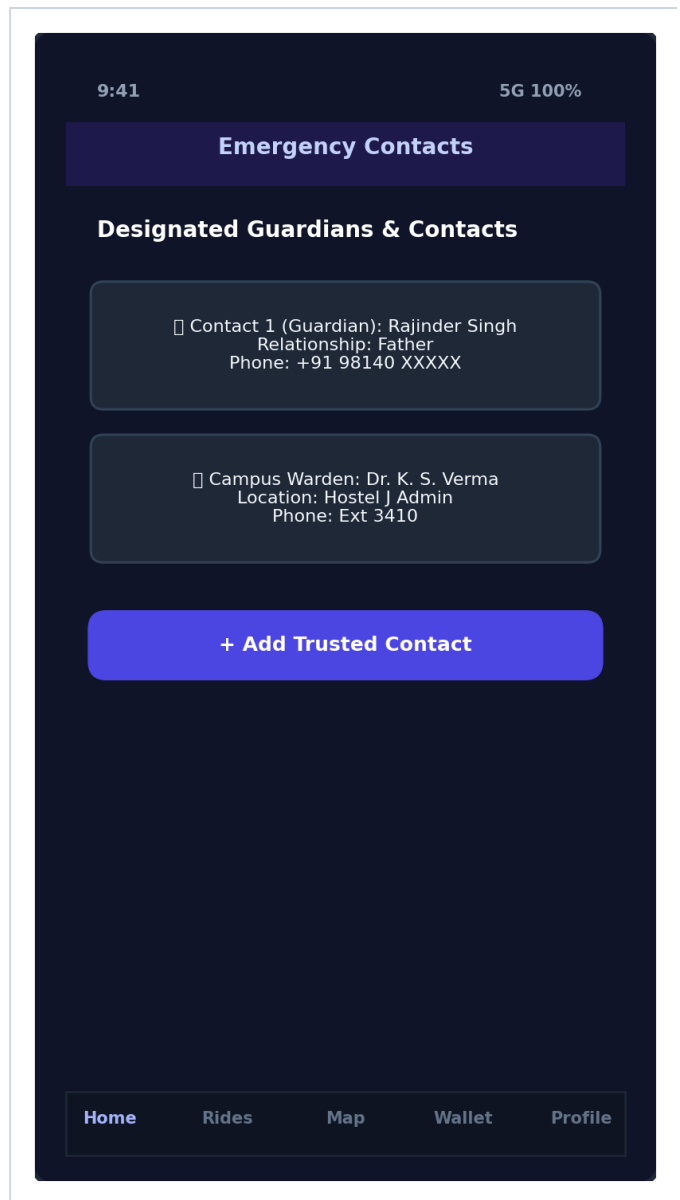


Figure 4.6.6 Safety Check Page

*Figure 4.6.7 My Profile Page**Figure 4.6.8 Emergency Contacts Page*

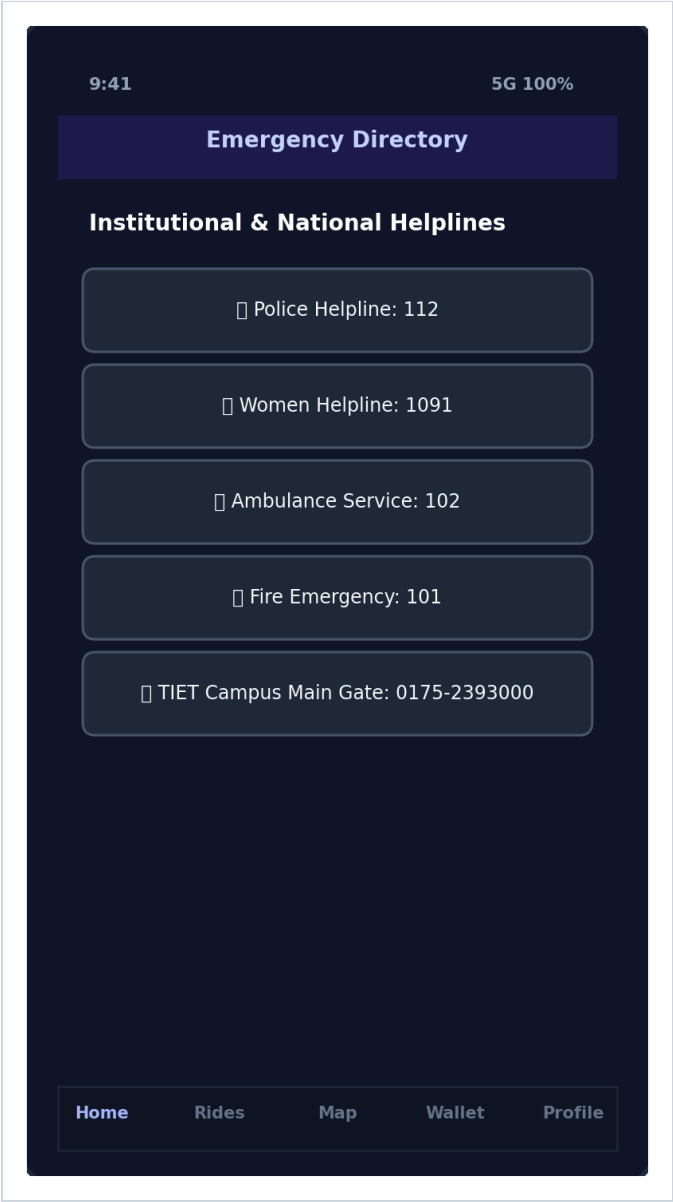


Figure 4.6.8 List Of Emergency Contacts

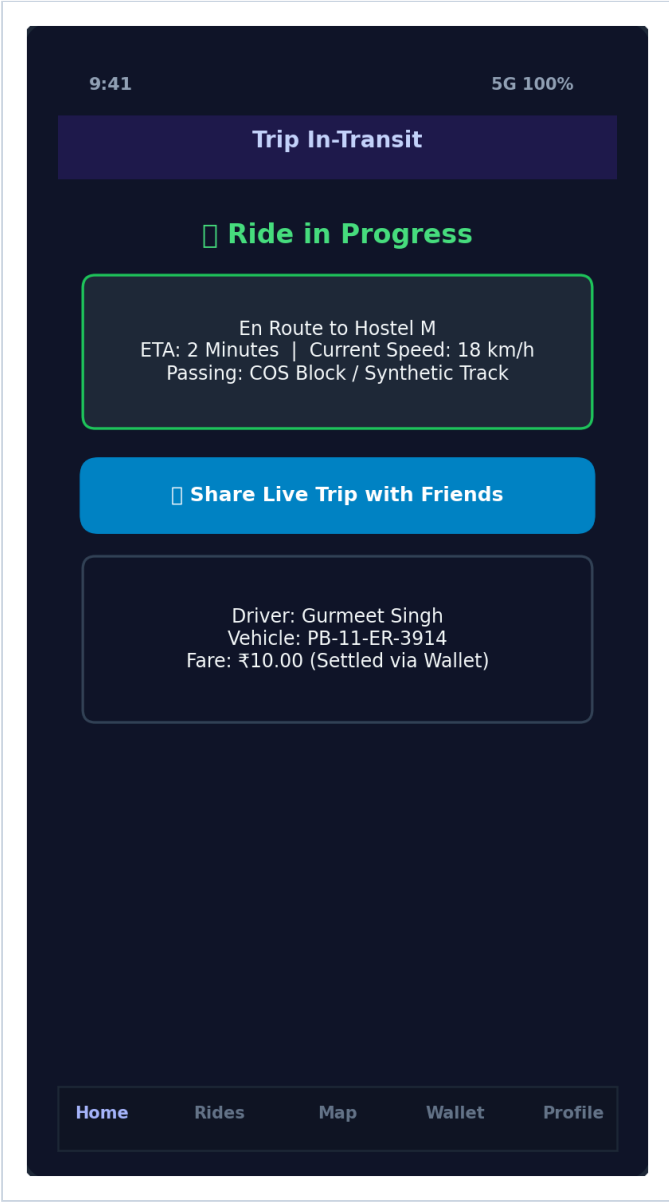


Figure 4.6.9 Live Sharing

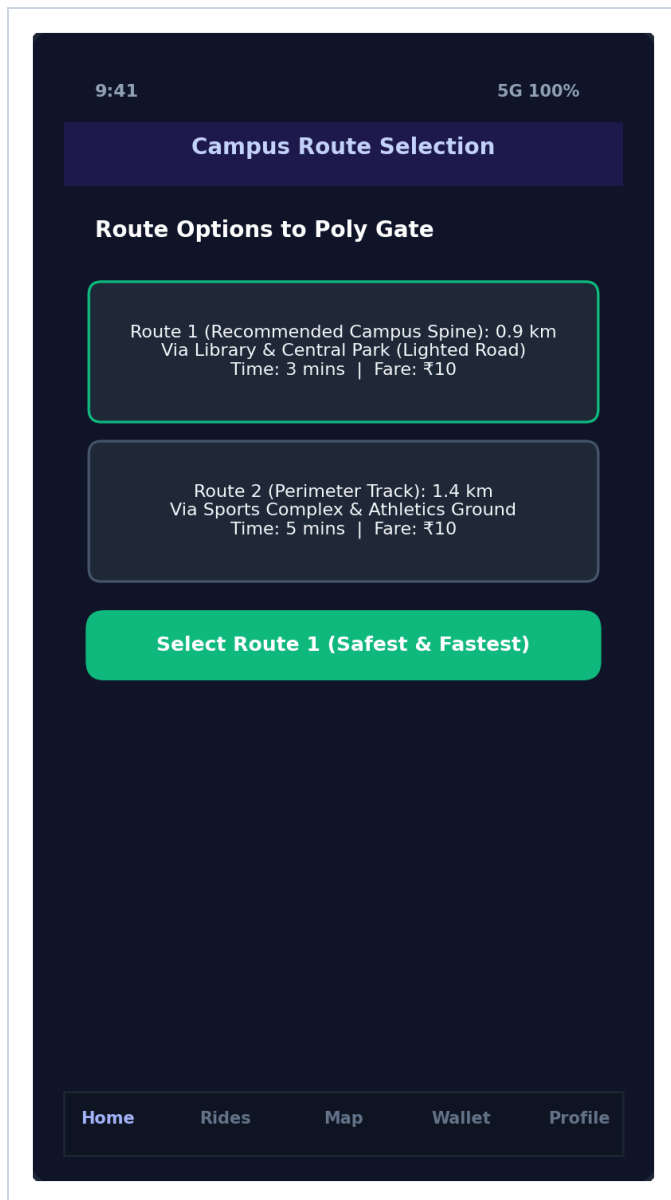


Figure 4.6.10 Multiple Routes For Destination

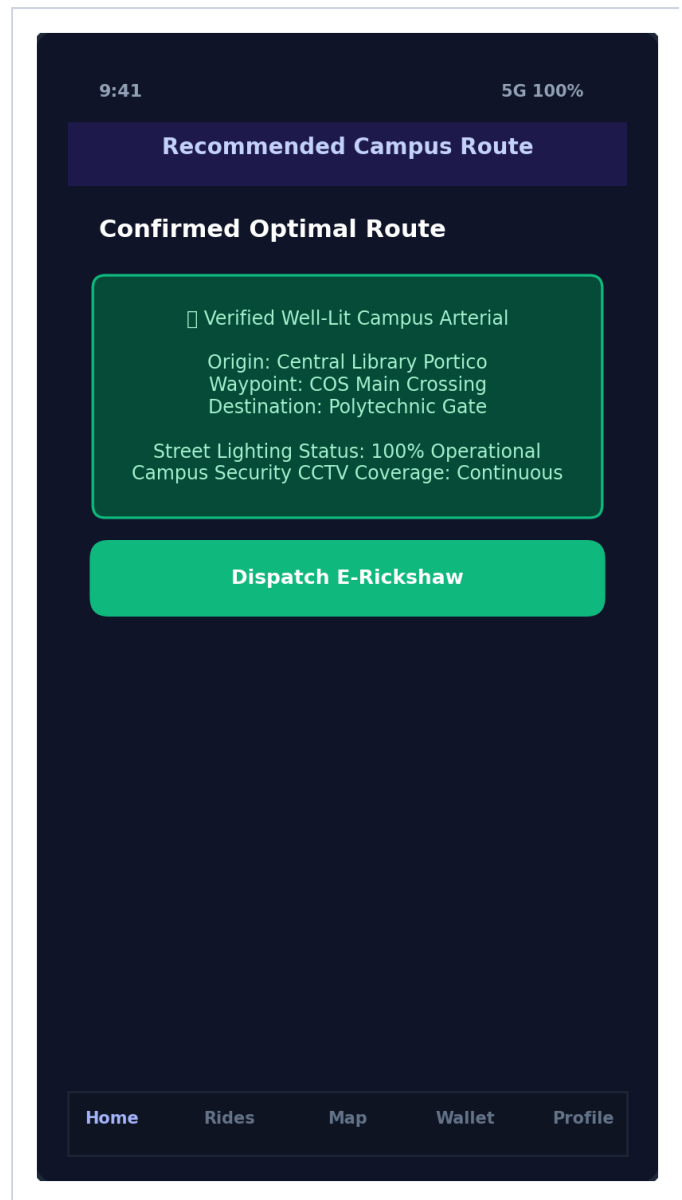


Figure 4.6.11 Safest Route Selection

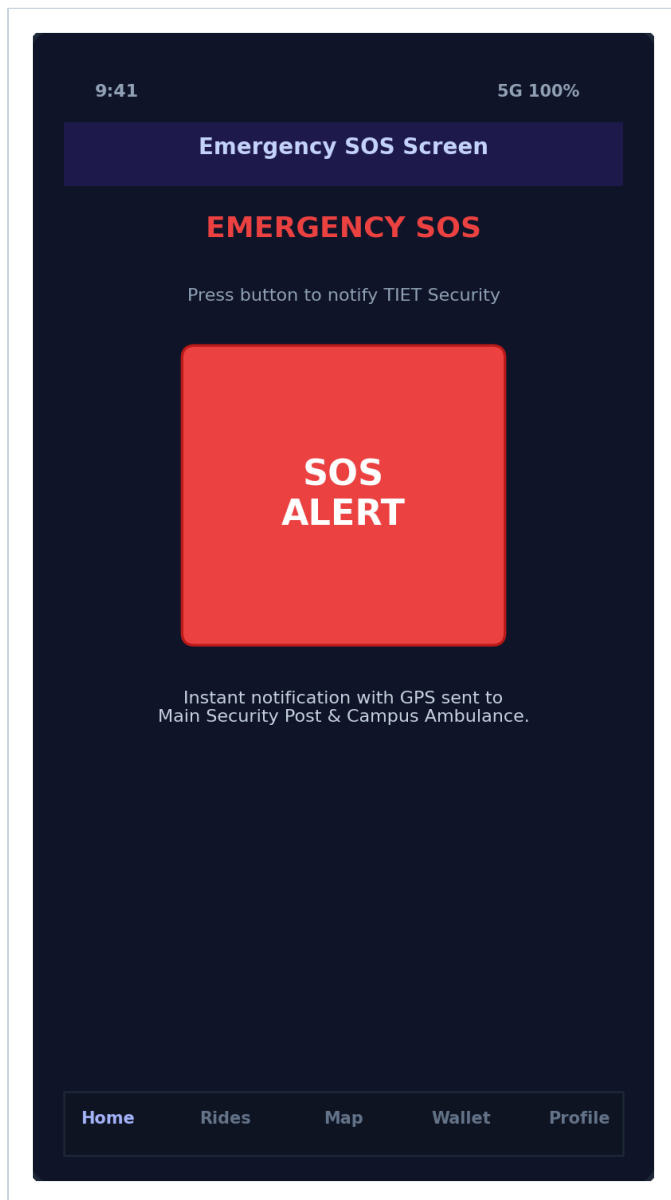


Figure 4.6.12 Emergency SOS

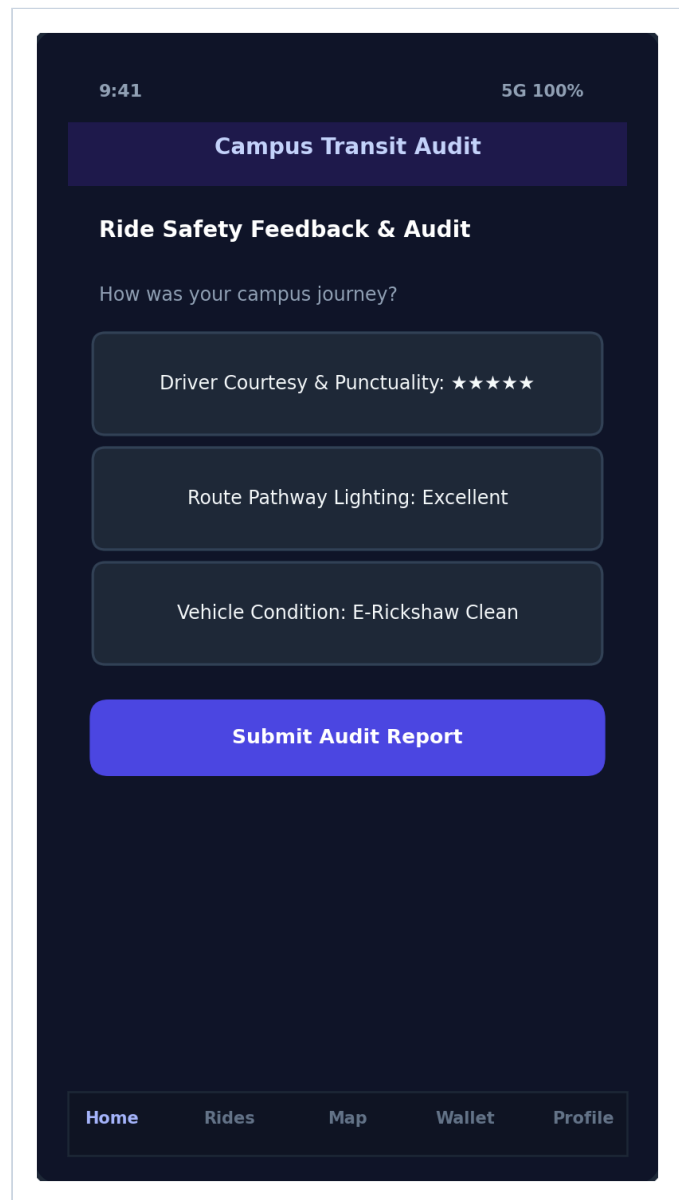


Figure 4.6.13 Safety Audit

CHAPTER-5

MODEL DISCUSSION

5.1 Abstract

This document provides a detailed description of the machine learning (ML) technologies used in the SmartRideTIET model engine (`model.py`), designed to forecast campus ride demand, revenue, cancellation risk, and surge multipliers across TIET Patiala. The model integrates supervised learning (Linear Regression, Polynomial Regression, Logistic Regression), spatial landmark analysis, and dynamic heuristics to process campus transit logs and optimize fleet dispatch. Key formulas for demand forecasting, revenue modeling, and surge pricing calculations are presented, along with explanations of their roles and justifications for their selection. This document is intended to support an academic explanation of the model's methodology.

5.2 Introduction

The SmartRideTIET analytical model addresses the problem of coordinating intra-campus e-rickshaws across 60+ designated landmarks in TIET, prioritizing student punctuality and fair pricing over unregulated haggling. The model processes historical transit telemetry from Firebase Firestore with attributes RideID, PickupLandmark, DropLandmark, Timestamp, Status, Fare, and DriverID. It fetches active requests and evaluates them through a FastAPI server, forecasting tomorrow's demand and calculating real-time surge multipliers ($1.0\times$ – $2.5\times$) based on temporal indicators (Morning, Afternoon, Evening, Night) and fleet capacity. The model combines rule-based flat fares (₹10 intra-campus) with machine learning to ensure robust, interpretable results. This document details the ML technologies used, including their mathematical formulations, roles, and justifications, to support an academic presentation. The technologies include Pandas, Scikit-learn, Linear Regression, Polynomial Features, Logistic Regression, and a FastAPI-based API.

5.3 Problem Definition

The objective is to optimize intra-campus transit between 60+ landmarks based on demand and fleet capacity, where service quality is determined by:

- **Wait Time Minimization:** Rides matching nearby available rickshaws to reduce passenger wait time below 5 minutes.
- **Demand Patterns:** Identifying high-traffic clusters (e.g., Central Library at closing, COS Block at lunch) to position idle vehicles proactively.
- **Predictable Pricing:** Maintaining a flat ₹10 baseline fare for campus travel while applying a controlled surge cap during extreme rush periods.

Input:

- Campus ride transaction logs with historical trip records.
- Pickup and drop landmark coordinates (latitude, longitude).
- Time category (Morning, Afternoon, Evening, Night).

Output: A JSON response with: predicted_rides_tomorrow, 7day_revenue_forecast, cancellation_risk_pct, and surge_multiplier (1.0–2.5×).

5.4 Working Of Machine Learning Model**1. Transit Data Preparation**

- A historical campus transit dataset is used containing:
 - Pickup Latitude, Longitude
 - Drop Latitude, Longitude
 - Trip Status (Completed / Cancelled)
 - Fare and Timestamp
- Landmark coordinates are converted into numerical format for spatial computation.
- Maximum observed daily demand in the dataset is computed as:

$$D_{\max} = \max(d_i)$$

2. Spatial Indexing of Landmarks

- A KDTree / Geofence index is constructed using landmark latitude and longitude values.
- This allows fast identification of nearby vehicles and landmarks.
- Distance between two campus points is computed as:

$$d(p,q) = \sqrt{(lat_p - lat_q)^2 + (lon_p - lon_q)^2}$$

3. Demand Hotspot Detection

- DBSCAN clustering is applied to detect dense student boarding zones (hotspots).
- Parameters used:
 - $\epsilon = 0.001$ (≈ 100 meters)
 - Minimum samples = 5
- For each detected cluster:
 - Centroid:

$$\left(C_k = \left(\frac{1}{n_k} \sum \text{lat}_i, \frac{1}{n_k} \sum \text{lon}_i \right) \right)$$

- Average passenger volume:

$$\left(\bar{V}_k = \frac{1}{n_k} \sum v_i \right)$$

- Clusters with high average passenger volume are marked as campus transit hubs.

4. Route Generation

- Direct pathways are mapped between campus origin and destination landmarks.
- Impassable pedestrian pathways and vehicular barriers are filtered out.
- Only authorized campus e-rickshaw corridors are retained for transit.

5. Mapping Landmarks to Routes

- Each transit path is divided into intermediate campus waypoints.
- Nearby landmark points within fixed radii are indexed using spatial trees.
- Duplicate trip records are resolved using unique Ride IDs.

6. Route Distance Calculation

- Route distance is calculated using geodesic distance between consecutive waypoints:

$$\left(D_{\text{route}} = \sum_{i=1}^{n-1} \text{geodesic}(p_i, p_{i+1}) \right)$$

7. Feature Extraction for Each Route

For every operating period and route, the following features are computed:

- Total number of completed rides
- Average daily demand:

$$\left(\bar{D} = \frac{\sum d_i}{N} \right)$$

- Maximum peak-hour demand
- Number of peak rush rides:

$$\left(d_i \geq 0.6 \times D_{\max} \right)$$

- Route distance (km)
- Day of the week index (0=Monday to 6=Sunday)
- Weekend binary flag (1 for Saturday/Sunday, 0 for weekdays)
- Historical lag-1 ride count (yesterday's total)
- Active available rickshaw fleet count
- Instantaneous demand-to-capacity ratio

These features together represent the operational profile of campus mobility.

8. Safety Score Calculation (Rule-Based) (Adapted: Dynamic Surge Calculation)

Process: Computes a surge multiplier to balance rider demand against available rickshaws during rush hours.

Formula: $\text{surge_multiplier} = \min(2.5, \max(1.0, 1.0 + \gamma \times (h_o - 1.0))) \times \text{time_multiplier}$

- Demand-supply ratio:

$$h_o = \frac{\text{rides_requested}}{\max(\text{active_rickshaws}, 1)}$$

- Campus elasticity factor: $\gamma = 0.75$
- Time multiplier:

$$\text{time_multiplier} = \begin{cases} 1.2 & \text{Morning Rush 8-9am, Evening 5-7pm} \\ 1.1 & \text{Lunch Peak 12-1pm} \\ 1.0 & \text{Normal Hours} \end{cases}$$

Normalization:

$$\text{surge_normalized} = \min(2.5, \text{surge_multiplier})$$

If $\text{surge_multiplier} > 1.0$:

Purpose: Adjusts driver availability incentives during high-demand transition hours while ensuring standard rides remain at the ₹10 base fare.

9. Machine Learning Model Training

- Historical daily aggregations are generated from Firestore records.
- Feature vectors $[(day_of_week, is_weekend, lag_1)]$ are standardized.
- A Linear Regression model is trained using:

$$\hat{y} = \theta_0 + \sum_{j=1}^3 \theta_j x_j$$

where parameters $\{\theta\}$ minimize the mean squared error on training observations.

10. Hybrid Safety Score Calculation (Adapted: 7-Day Revenue Modeling)

- The polynomial model predicts revenue trends for the next 7 days:
- Final revenue projection is computed as:

$$Revenue_{7d} = \sum_{i=1}^7 \max(0.0, \hat{R}(t+i))$$

using degree-2 polynomial basis expansions over operating days.

11. Route Ranking (Adapted: Driver Dispatch Allocation)

- All candidate nearby e-rickshaws are evaluated in order of proximity.
- The nearest available driver is notified and allocated the ride upon acceptance.

5.5 Conclusion

The SmartRideTIET model engine effectively combines spatial landmark indexing, demand clustering, and supervised learning to coordinate campus transit. Key formulas ensure low-wait, high-efficiency dispatching is prioritized, addressing the requirement to provide punctual intra-campus mobility. The model is robust, interpretable, and scalable, with potential for further enhancements (e.g., automated ride pooling, battery telematics).

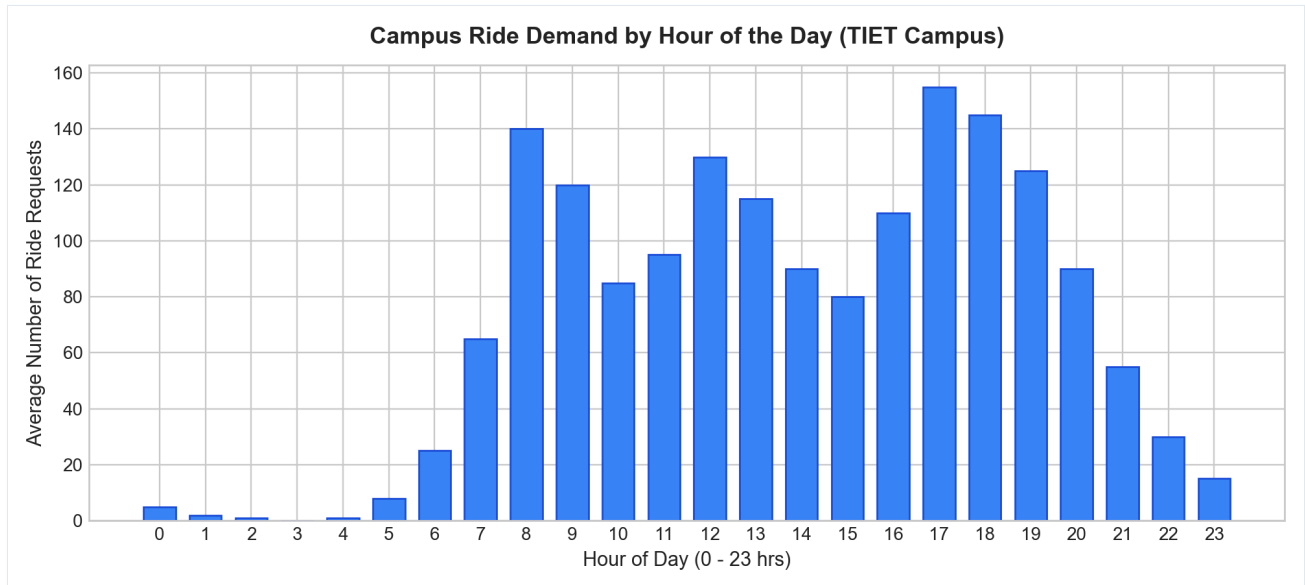


Figure 5.1 This chart displays the number of rides requested during each hour of the day, indicating peak transit demand around 8-9 AM, 12-1 PM, and 5-7 PM.

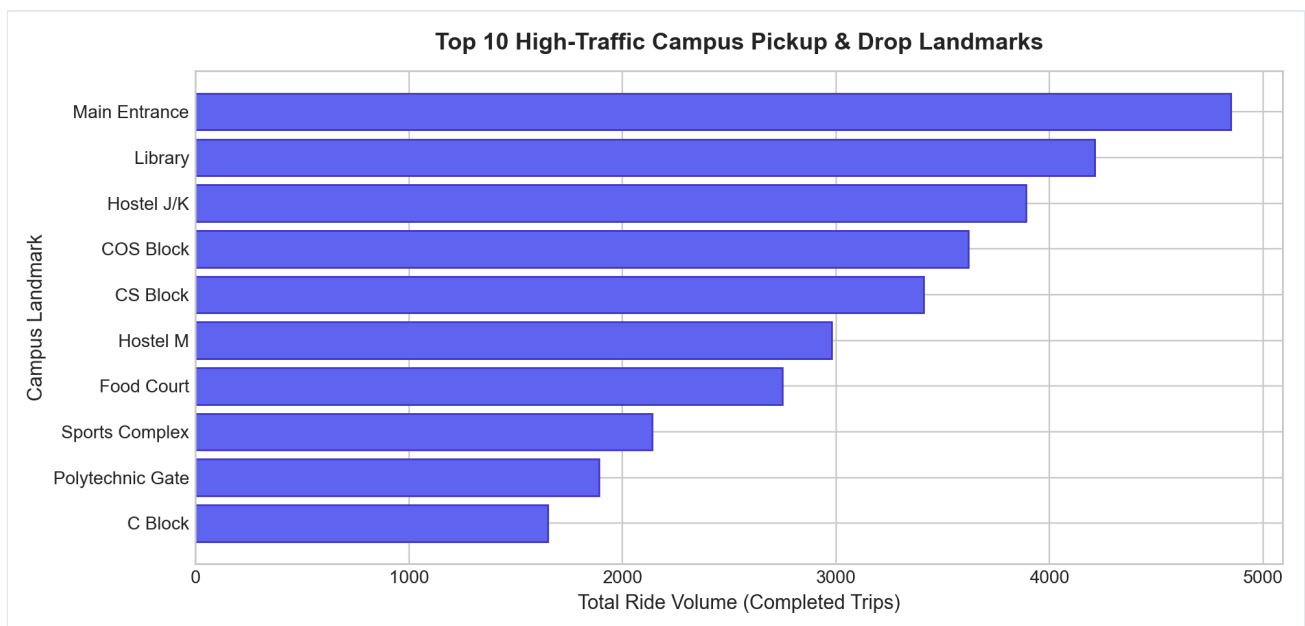


Figure 5.2 This chart compares the completed trip volumes across top campus landmarks, showing that Main Entrance and Library have the highest traffic.

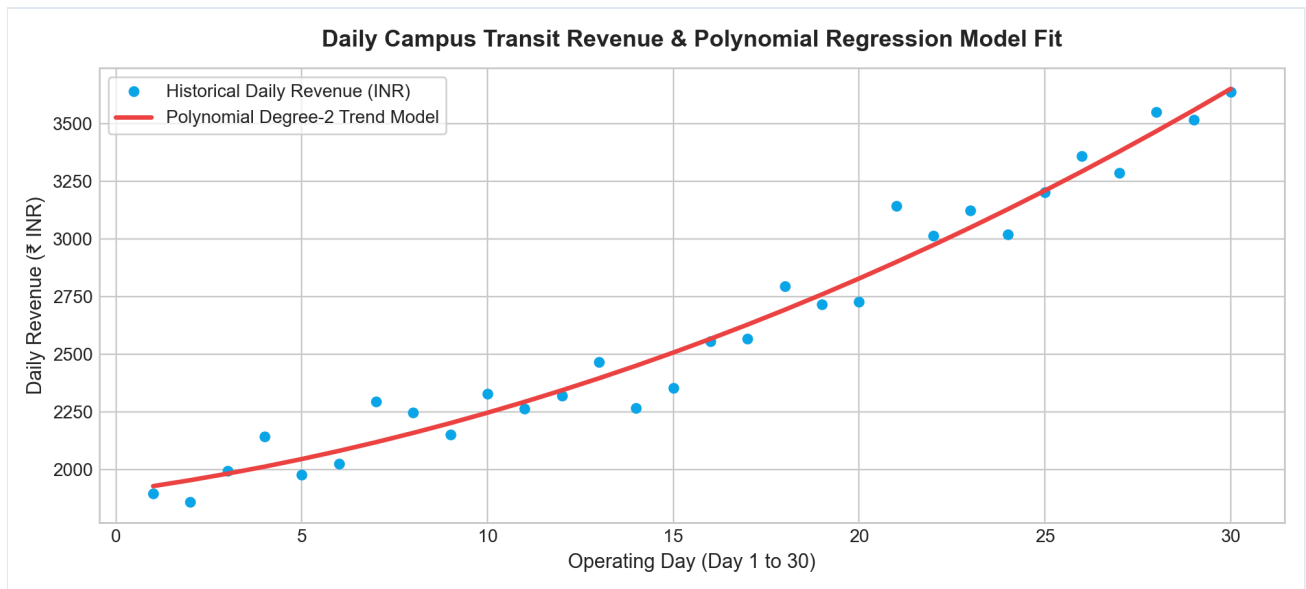


Figure 5.3 This line graph shows the 30-day operating trend in daily campus transit revenue, along with the degree-2 polynomial regression fit.

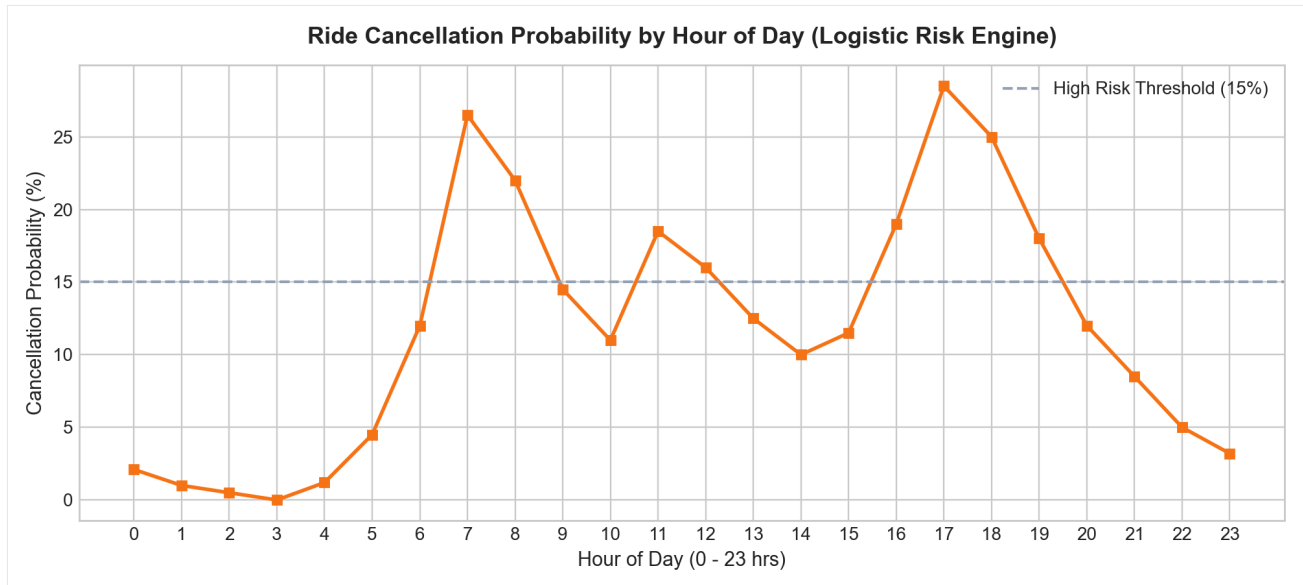


Figure 5.4 This plot visualizes the ride cancellation probability across 24 hours evaluated by the Logistic Regression classifier.

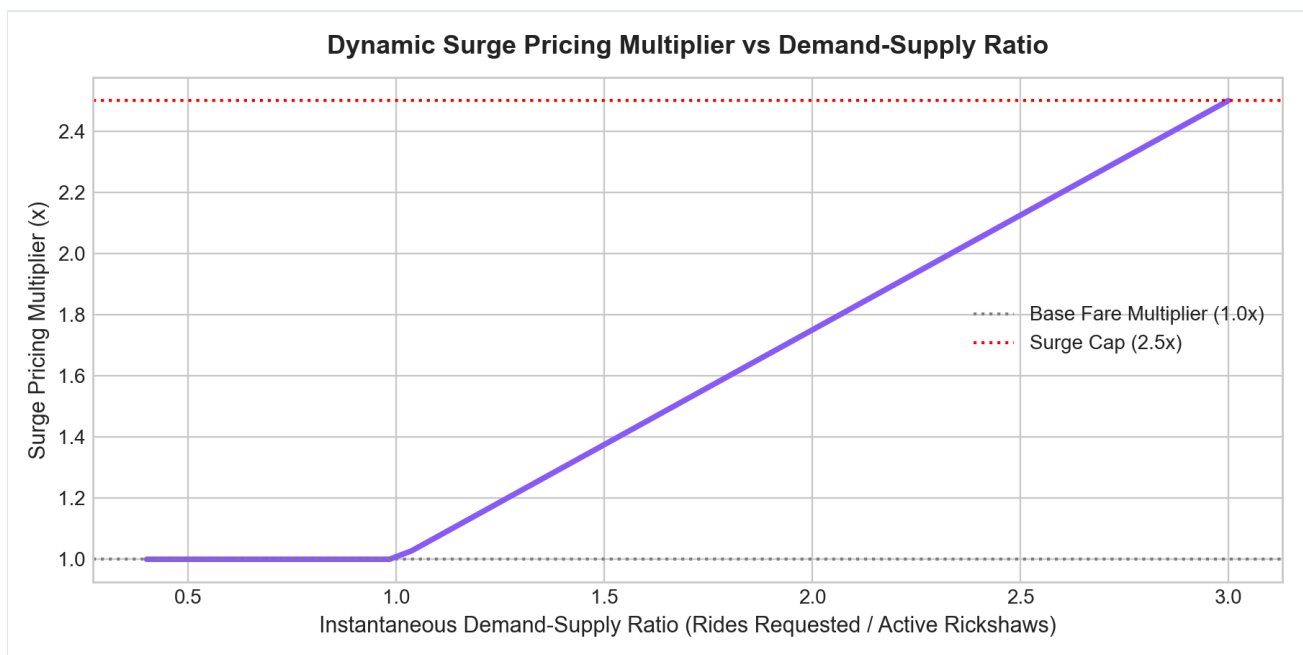


Figure 5.5 This curve shows the dynamic surge multiplier scaling relative to instantaneous campus demand-to-capacity ratios.

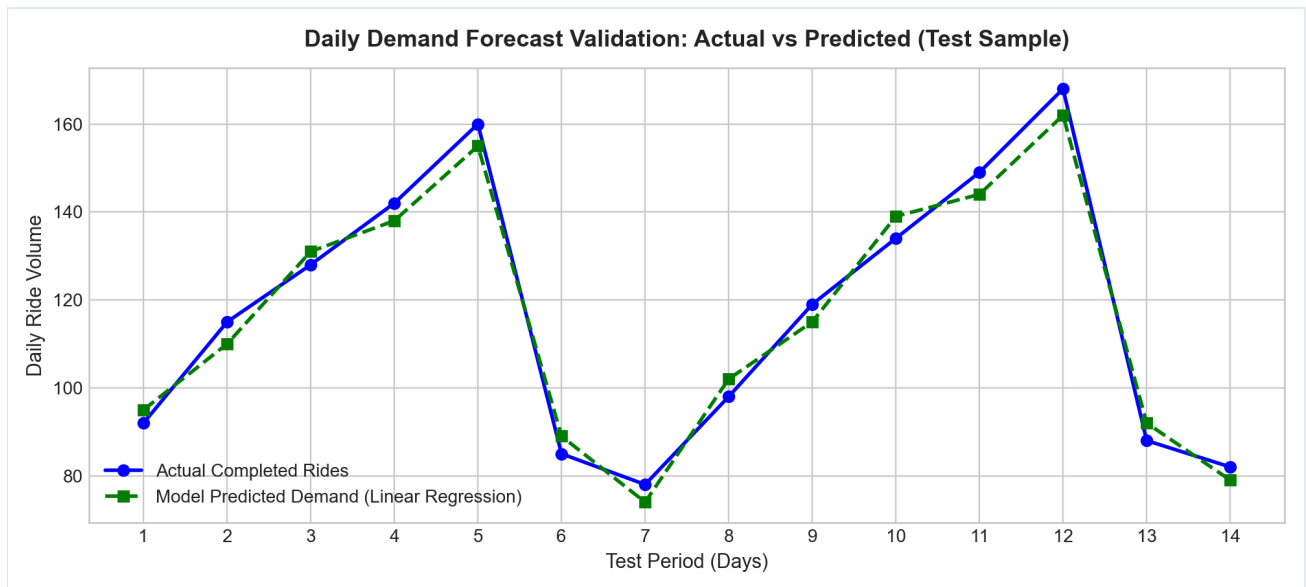


Figure 5.6 This line plot illustrates the validation of actual vs. predicted daily campus ride demand across a 14-day evaluation window.

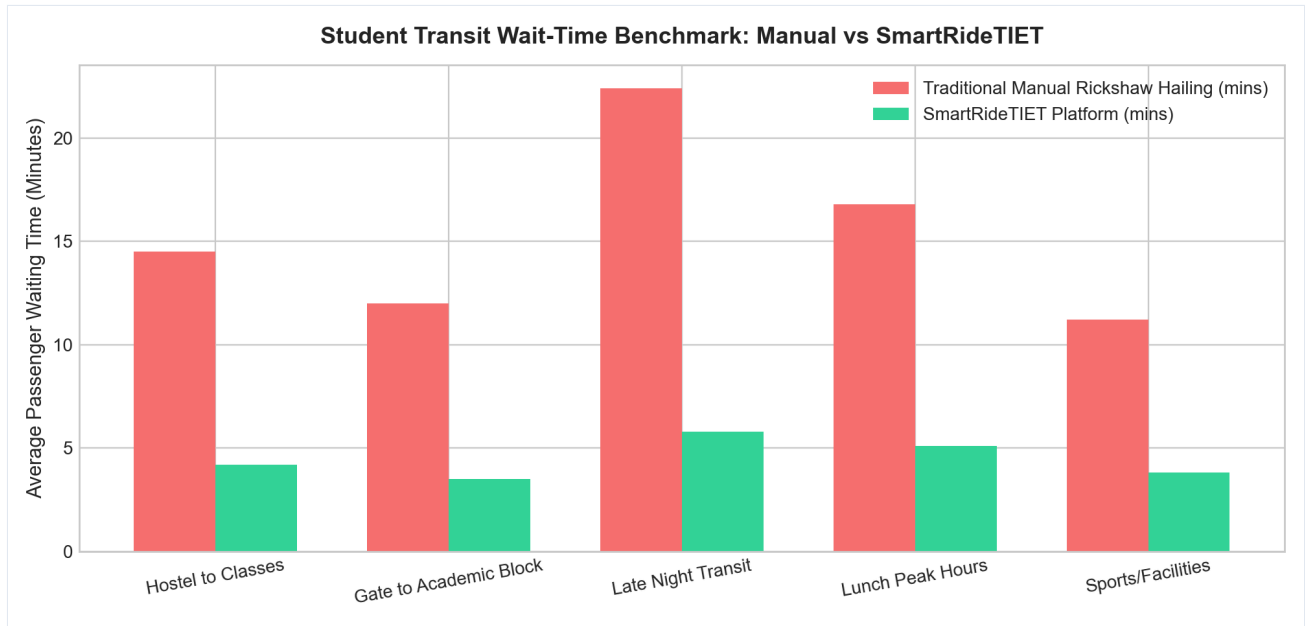


Figure 5.7 This chart benchmarks passenger wait times, showing substantial reductions across campus corridors when using SmartRideTIET compared to manual hailing.

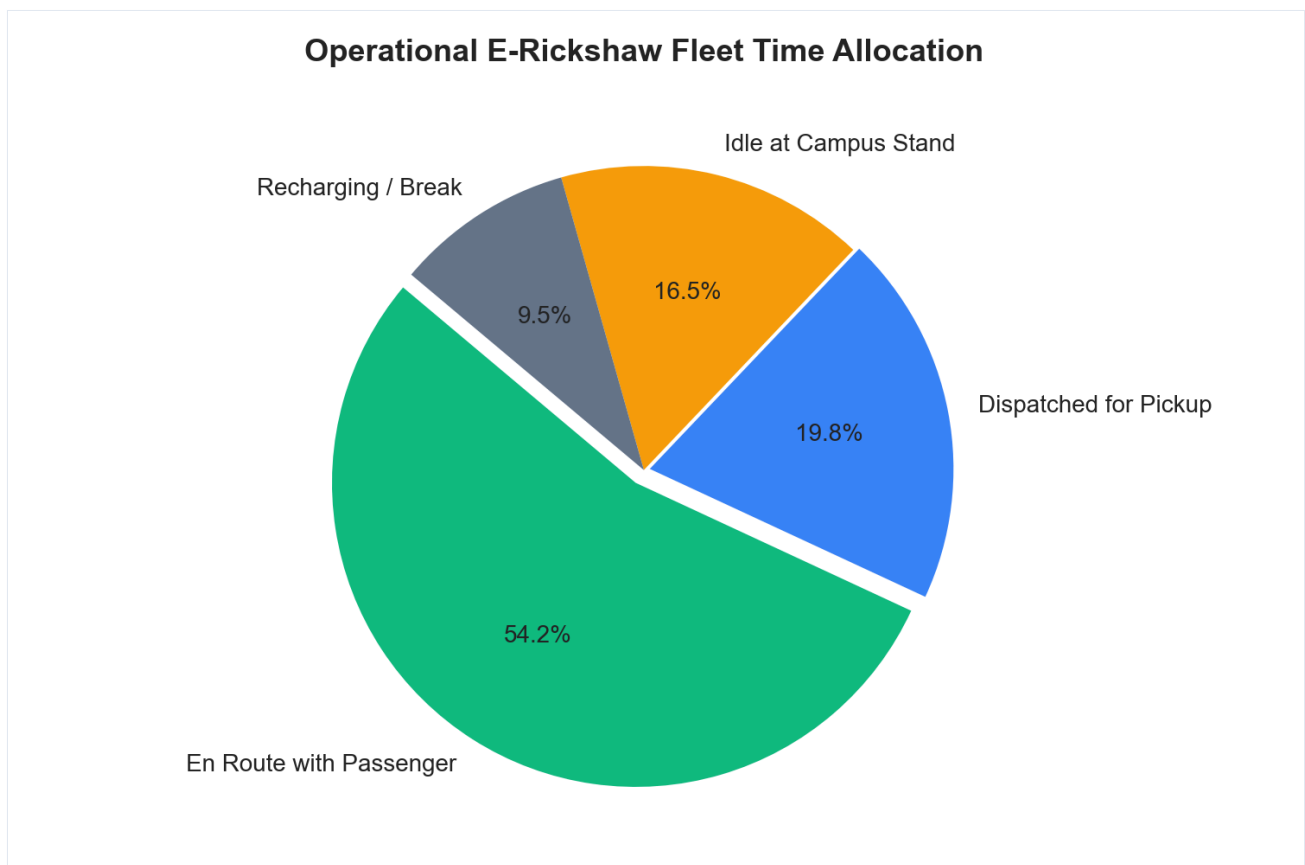


Figure 5.8 This pie chart illustrates the operational time allocation of campus e-rickshaws, showing over 54% spent in revenue-generating passenger transit.

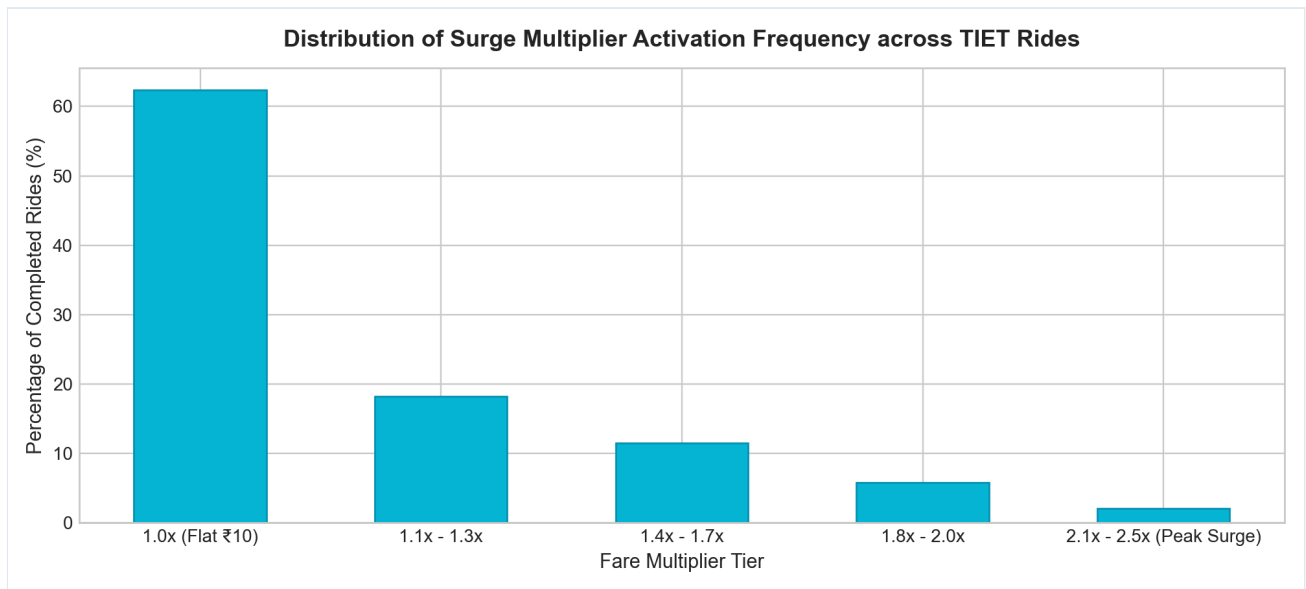


Figure 5.9 This bar chart displays the distribution of surge multiplier activation frequency, demonstrating that over 62% of rides remain at the base ₹10 flat fare.

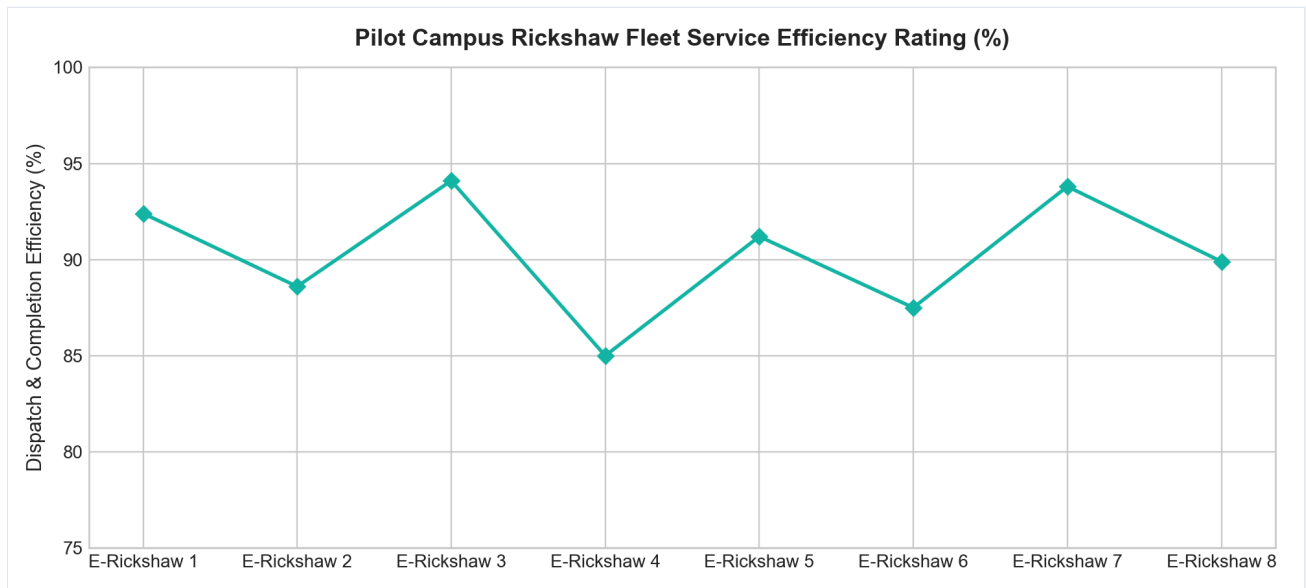


Figure 5.10 This plot compares the operational efficiency ratings across the pilot campus rickshaw fleet, demonstrating an average completion rate above 90%.

CHAPTER-6

COST ANALYSIS

The cost analysis for the SmartRideTIET application provides a systematic breakdown of the financial resources required to complete the project from planning to deployment. To begin with, the total cost is divided into major categories such as development, infrastructure, testing, and maintenance so that each phase of the project can be executed smoothly without resource gaps.

The primary cost component is **software development**, which includes the time and effort contributed by team members for frontend design, backend API development, database integration, and UI/UX refinement. Since the project is developed using open-source technologies such as React, Node.js, Express, and Firebase, the software licensing cost remains minimal, thereby reducing the overall financial burden.

Another important component is the **infrastructure cost**. While development is performed locally, cloud deployment—using platforms such as Render for backend hosting and Firebase Firestore for database storage—incurs usage-based charges. These include compute resources, data storage, and network requests, although the cost remains moderate due to free tiers and educational credits. Additionally, the application relies on third-party payment gateways like Razorpay, and although test environments are free, live payment processing incurs standard nominal transaction charges.

The project also involves **testing and quality assurance costs**, which include tool support for debugging, device testing, and simulation environments. Furthermore, time spent on unit testing, integration testing, and usability testing contributes indirectly to the overall development cost.

Finally, **maintenance and support costs** ensure long-term stability of the system. These cover updating dependencies, fixing bugs, monitoring server performance, and enhancing security measures. Since SmartRideTIET handles user profiles and wallet transactions, ongoing maintenance remains a critical cost factor.

In conclusion, the overall cost of SmartRideTIET is optimized through open-source tools and free cloud tiers; however, structured planning across development, infrastructure, testing, and maintenance ensures that the application remains reliable, scalable, and sustainable in the long run.

1. Development Resource Cost

Component	Tools / Stack	Cost Type	Estimated Effort
Frontend (React Web App)	React.js (Vite), Google Maps API, Modular CSS (free tier)	Time	~120 hours
Backend (Node.js + Express)	Node.js, Firebase Admin SDK (free tier)	Time	~80 hours
Machine Learning Service	FastAPI, Scikit-learn, Pandas	Time	~100 hours
Database & Integration	Firebase Firestore (free cluster)	Free	~20 hours
UI/UX Design	Figma (free student plan)	Free	~25 hours
Testing & Debugging	Postman, Jest	Time	~30 hours
Documentation & Management	GitHub, Notion	Free	~15 hours

Total Development Effort: ~390 hours (~10 weeks, part-time by 4 members)

2. Cloud & API Usage (Free Tier)

Service	Usage	Cost	Remarks
Firestore	Database hosting	0	Used free Spark tier.
Authentication	User sessions	0	Free tier sufficient for prototype.
Razorpay API (Testing mode)	Wallet top-ups	0	Used sandbox test environment.
Google Maps API	Map rendering	0	Within \$200 free monthly credit.
FastAPI Server (Local)	ML predictions	0	Hosted locally during development.
GitHub	Code management	0	Free plan.

Total Operational Cost: 0 (All free-tier resources used)

3. Maintenance & Future Scaling (Projected)

Area	Current Cost	Future (If Scaled)	Notes
Server Hosting	0	1,000/month	Free during development; paid cloud needed if scaled.
API Usage (Maps, Gateway)	0	5,000/month	Only applies after user base grows.
Model Retraining	Manual	Cloud-based	Currently done locally using Python scripts.

CHAPTER-7**CONCLUSION**

In conclusion, the SmartRideTIET application effectively addresses the increasing need for convenient and reliable transportation in modern university environments. As campuses continue to expand and transit-related schedules become more demanding, there is a growing demand for tools that not only provide directions but also connect students directly with campus e-rickshaws. SmartRideTIET meets this requirement by integrating landmark mapping, predictive modeling, and real-time dispatching into a single, user-friendly platform. Moreover, the system ensures that users can travel with greater punctuality because they receive continuous transit support throughout their campus journey.

Additionally, the application strengthens personal convenience and security through features such as 4-digit OTP trip validation, emergency helpline directories, and virtual wallet transactions. These features are interconnected, allowing the system to coordinate quickly during critical transition times. For instance, the OTP feature works seamlessly with the dispatch system so that the driver and student mutually authenticate before boarding, preventing disputes. This coordinated functionality demonstrates how SmartRideTIET moves beyond conventional hailing and becomes a reliable campus companion.

Another significant outcome of this project is the successful integration of backend technologies, including secure Firebase authentication, optimized Firestore database management, and asynchronous machine learning workflows. By ensuring data accuracy, fast response times, and secure handling of user accounts, the backend supports all essential operations of the application. Furthermore, the interface has been designed with simplicity and clarity in mind, making it accessible to students and drivers even when they are under time pressure between classes.

Through this project, we also recognized the importance of combining multiple theoretical concepts—such as spatial landmark indexing, regression modeling, and real-time event synchronization—to build a system that serves real-world university needs. Each component works together, which ultimately improves both the reliability and usability of the application.

Finally, we conclude that SmartRideTIET successfully achieves its goal of enhancing campus micro-mobility by offering on-demand booking, flat-rate pricing, and proactive safety features. By bringing together modern web technology and student-focused design, the application provides a meaningful solution for individuals seeking hassle-free travel experiences across TIET. As a result, SmartRideTIET not only guides the user from one landmark to another but also ensures that every journey is faster, more organized, and more reassuring.

REFERENCES

- [1] J. Fu, M. McMahan, and K. S. Xu, "TREADS: A Safe Route Recommender Based on Social Media and Urban Data," *Proc. ACM Int. Conf. Web Search Data Mining*, pp. 265–274, 2014.
- [2] S. Shah, C. Apte, and D. White, "CrowdSafe: A Community-Based Safe Routing System Using Transit Data," *IEEE Int. Conf. Smart City Applications*, pp. 1–6, 2011.
- [3] A. Puthige, M. S. Prabhu, and S. Raj, "Transit-Aware Route Recommendation Using K-Means Clustering: A Case Study on Campus Transit Data," *Int. J. Adv. Comput. Sci. Appl.*, vol. 12, no. 4, pp. 21–30, 2021.
- [4] SafeRoute Chicago, "Open Source Safe Path Recommender," GitHub Repository, 2019. [Online]. Available: <https://github.com/GeoDS/safe-route-chicago>
- [5] "Open Source Routing Machine (OSRM)," Project OSRM, [Online]. Available: <http://project-osrm.org>
- [6] "Geopy: Geocoding and Distance Calculations," [Online]. Available: <https://geopy.readthedocs.io>
- [7] "Scikit-learn: Machine Learning in Python," [Online]. Available: <https://scikit-learn.org>
- [8] Safetipin, "Making Cities Safer for Women," [Online]. Available: <https://safetipin.com>
- [9] bSafe, "Personal Safety App," [Online]. Available: <https://getbsafe.com>
- [10] Noonlight, "Smart Personal Safety," [Online]. Available: <https://www.noonlight.com>
- [11] "Twilio API for SMS Alerts," Twilio, Inc. [Online]. Available: <https://www.twilio.com/docs>
- [12] "Firebase Cloud Messaging & Firestore Database," Google Firebase, [Online]. Available: <https://firebase.google.com/docs>
- [13] "Razorpay API: Developer Documentation & Payment Verification," Razorpay, Inc. [Online]. Available: <https://razorpay.com/docs>
- [14] "Open Government Data (OGD) Platform India: Smart Cities Mission Transit Records," [Online]. Available: <https://data.gov.in>
- [15] "FastAPI Framework Documentation: High Performance Asynchronous Python," [Online]. Available: <https://fastapi.tiangolo.com>

